

RAPPORT DE STAGE

FELLAH ZAKARIYA

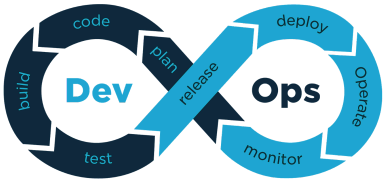
Stagiaire DevOps & Cloud AWS

BTS Services Informatiques aux Organisations - Option SISR

Stage effectué du 11 Mai 2026 au 3 Juillet 2026 au sein de **CONSULTAS GROUP**

Sous la direction de:

M.BELBACHA SAMI





Année Scolaire 2025 - 2026

Remerciements

Je tiens tout d'abord à remercier **M.Belbacha**, mon tuteur de stage au sein de Consultas Group, pour le temps qu'il m'a consacré tout au long de ces huit semaines.

Malgré sa propre activité professionnelle en parallèle, il a toujours su se rendre disponible pour m'expliquer des notions complexes, revenir patiemment sur mes difficultés techniques, et répondre à mes nombreuses questions, parfois lors de réunions techniques s'étendant sur trois à quatre heures.

Je le remercie particulièrement pour sa patience et sa pédagogie, qui m'ont permis de progresser à mon rythme sans jamais me sentir dépassé, ainsi que pour la confiance qu'il m'a accordée en me laissant une réelle autonomie dans la conduite de mes missions.

Je remercie également **M.Miloudi**, fondateur et président de Consultas Group, pour m'avoir accueilli au sein de son entreprise et permis de réaliser ce stage dans de bonnes conditions.

Je remercie également l'ensemble de l'équipe pédagogique du BTS Services Informatiques aux Organisations du Lycée Turgot, pour la formation théorique qui m'a permis d'aborder ce stage avec des bases solides.

Enfin, je remercie toute personne ayant, de près ou de loin, contribué à la réussite de ce stage et à la rédaction de ce rapport.

Sommaire

Introduction **1**
1. Présentation de l'entreprise **2**
2. Missions réalisées **3**
2.1 Missions 1 à 6 - Apprentissage et développement de l'application Pokédex 3
2.1.1 Mission 1 - Découverte des fondamentaux DevOps 3
2.1.2 Mission 2 - Approfondissement Git/GitHub et introduction CI/CD 4
2.1.3 Mission 3 - Consolidation des acquis Git, GitHub et CI/CD 5
2.1.4 Mission 4 - Développement de l'application Pokédex 5
2.1.5 Mission 5 - Finalisation de l'application Pokédex et mise en place de la CI 6
2.1.6 Mission 6 - Test de déploiement avec Coolify sur VirtualBox 8
2.2 Mission 7 - Migration vers une infrastructure cloud AWS 9
2.2.1 - Semaine 7 - Déploiement initial sur AWS 9
A. Sécurisation du compte et mise en place de l'infrastructure de calcul 10
B. Mise en place d'une base de données managée avec Amazon RDS 11
C. Déploiement de l'application via Coolify 12
D. Exploration du réseau AWS et mise en place du stockage S3 13
2.2.2 Semaine 8 - Sécurisation et amélioration de l'infrastructure 14
A. Sécurisation des communications avec HTTPS 14
B. Mise en place du monitoring avec CloudWatch 15
C. Automatisation du stockage avec Amazon S3 16
D. Documentation technique et finalisation 17
3. Compétences techniques acquises **18**
4. Compétences professionnelles développées **21**
5. Bilan personnel **23**
Ce que ce stage m'a apporté 23
Difficultés rencontrées 23
Ce que je ferais différemment 24
Perspectives 24
Conclusion **25**
Glossaire **26**
Annexes **30**

Introduction

Depuis l'apparition du cloud computing, de plus en plus d'entreprises font le choix de migrer une partie, voire l'intégralité de leur infrastructure vers le cloud. Cette transformation s'accompagne également d'une évolution des pratiques de développement, avec l'adoption croissante de méthodologies DevOps visant à automatiser et fiabiliser les cycles de développement et de déploiement. J'ai voulu comprendre pourquoi ces entreprises s'intéressent autant à ces technologies.

Dans le cadre de mon BTS Services Informatiques aux Organisations, j'ai choisi d'effectuer mon stage, du 18 mai au 3 juillet 2026, chez Consultas Group, une petite structure spécialisée dans le développement d'applications et les migrations vers le cloud.

Au sein de cette entreprise, j'ai occupé le poste de Stagiaire DevOps & Cloud AWS, notamment chargé du développement d'un pipeline d'automatisation des tests d'intégration (CI) et de déploiement (CD), ainsi que de la migration d'une application vers le cloud AWS.

Ce rapport présente dans un premier temps l'entreprise qui m'a accueilli, puis détaille l'ensemble des missions réalisées durant ce stage, avec un focus particulier sur la migration cloud qui en constitue le point culminant, avant de dresser un bilan des compétences techniques et professionnelles acquises.

1. Présentation de l’entreprise

Consultas Group est une société par actions simplifiée (SAS) créée le 2 janvier 2024, basée à Houilles, avec un capital social de 1000 euros. Son activité principale, référencée sous le code NAF 62.02A « Conseil en systèmes et logiciels informatiques », couvre un large éventail de prestations informatiques et numériques : conseil, expertise, conception, développement, maintenance, intégration et formation dans les domaines de l'informatique, des systèmes d'information et des nouvelles technologies.

Dans les faits, l'entreprise est spécialisée dans le développement d'applications ainsi que dans la migration d'applications et d'infrastructures vers le cloud, notamment sur les plateformes AWS et OVHcloud.

L'équipe de Consultas Group est composée de deux personnes : Fouad, fondateur et président de la société, également Project Manager chez Mitsubishi Electric et développeur full-stack, également spécialisé dans les technologies DevOps, ainsi que mon tuteur de stage, Sami, Tech Lead Backend spécialisé en PHP/Symfony et architecture logicielle, également ingénieur logiciel senior chez Driveco.

Bénéficiant de l'expertise de mon tuteur de stage sur l'architecture logicielle, ainsi que des compétences DevOps de Fouad, j'ai pu être accompagné et guidé tout au long de mon stage, notamment sur la migration cloud qui constitue le cœur de ce rapport.

Voir Annexe 2

2. Missions réalisées

Durant mon stage, j'ai eu l'opportunité de participer à plusieurs missions progressives, me permettant de monter en compétences aussi bien sur le développement applicatif que sur l'infrastructure cloud.

2.1 Missions 1 à 6 - Apprentissage et développement de l'application Pokédex

Les cinq premières semaines de mon stage ont été consacrées à l'acquisition progressive des compétences théoriques et pratiques indispensables à mon domaine d'intervention : le développement d'API, la conteneurisation avec Docker, la gestion de versions avec Git et GitHub, ainsi que la mise en place de pipelines d'intégration continue (CI).

Cette montée en compétences progressive m'a permis de concevoir et déployer, en fin de période, une application full-stack complète: le Pokédex, qui constituera le fil conducteur du reste de ce rapport.

2.1.1 Mission 1 - Découverte des fondamentaux DevOps

Cette première semaine a été consacrée à l'acquisition des bases théoriques et techniques nécessaires à mon environnement de travail DevOps. J'ai découvert le fonctionnement des API REST, ainsi que le framework FastAPI en Python, accompagné des modules JSON, Typing et Dataclasses, à travers la réalisation d'un petit projet API personnel.

Voir Annexe 3.1 et 3.2

J'ai également débuté mon apprentissage de **Docker**, en manipulant mes premiers conteneurs et images, avant de concevoir mon propre Dockerfile ainsi qu'un fichier **docker-compose.yml** orchestrant plusieurs services.

Enfin, je me suis formé aux mécanismes de chiffrement et d'authentification SSH (clés symétriques et asymétriques), une compétence essentielle pour sécuriser les accès aux futurs serveurs de déploiement.

2.1.2 Mission 2 - Approfondissement Git/GitHub et introduction CI/CD

Cette deuxième semaine a été davantage axée sur l'apprentissage théorique, avec l'étude des concepts fondamentaux liés aux pipelines CI/CD (Continuous Integration / Continuous Deployment).

Suite à une démonstration de mon tuteur de stage sur le déploiement d'une application via un pipeline CI/CD sur un VPS OVHcloud, j'ai approfondi ma pratique de Git et GitHub : gestion des branches, merge, génération de clés SSH d'authentification, et création d'un token d'accès pour travailler depuis ma machine locale.

J'ai également repris mon API FastAPI développée la semaine précédente afin de tester ses endpoints avec **Postman**, et créé un **Makefile** simplifiant la gestion des containers Docker via des alias.
Voir Annexe 4.1 et 4.2

Cette semaine s'est conclue par une première tentative de rédaction d'un fichier de configuration CI (**ci.yml**), ainsi qu'un approfondissement de la notion de branches et de fusion (merge) sous Git, concept qui avait initialement posé quelques difficultés de compréhension.

2.1.3 Mission 3 - Consolidation des acquis Git, GitHub et CI/CD

Cette troisième semaine avait pour objectif de consolider les acquis des deux semaines précédentes, en retravaillant les notions déjà vues: Git, GitHub et les pipelines CI/CD; afin de corriger certaines lacunes avant d'aborder de nouvelles notions.

J'ai repris le fonctionnement global des pipelines CI/CD ainsi que le rôle de Git dans un environnement de développement collaboratif, avant d'approfondir la logique des branches et leur utilité dans l'organisation d'un projet.

J'ai également retravaillé la notion de **merge**, qui avait posé quelques difficultés initiales, ainsi que la structure d'un fichier CI (ci.yml), notamment la définition des jobs et des étapes d'exécution.

Cette semaine s'est conclue par une tentative de reconstruction autonome d'un fichier de pipeline CI, permettant d'identifier les points restants à consolider avant d'aborder la mise en pratique complète la semaine suivante.

2.1.4 Mission 4 - Développement de l'application Pokédex

Cette quatrième semaine de stage a été consacrée à la mise en pratique de l'ensemble des notions théoriques acquises durant les trois semaines précédentes, à travers le développement complet d'une application : le backend avec le framework **FastAPI**, la conteneurisation avec **Docker**, l'orchestration avec **Docker Compose**, ainsi que la gestion du versioning avec **Git/GitHub**.

Après une réunion de cadrage avec mon tuteur de stage définissant les objectifs de la mission, j'ai développé l'API en repartant des bases construites précédemment, en corrigeant notamment une méthode GET afin qu'elle récupère les données directement depuis la base de données plutôt que depuis un fichier JSON statique.
J'ai ensuite initié un dépôt Git local et mis en place l'ensemble des fichiers structurants du projet: le **Dockerfile**, le **docker-compose.yml** pour l'orchestration des services, un **Makefile** pour simplifier les commandes courantes, ainsi que les fichiers applicatifs (app.py pour le backend, index.html pour le frontend).

Le reste de la semaine a été consacré à l'affinement de la configuration Docker, notamment la résolution de problèmes de communication entre les différents conteneurs (backend, base de données, frontend), dans l'objectif de rendre l'environnement de développement totalement fonctionnel et reproductible. Cette semaine s'est achevée sans que l'ensemble du projet soit totalement finalisé, certains éléments comme le pipeline CI restant encore à développer.
Voir Annexe 5.1 et 5.2

2.1.5 Mission 5 - Finalisation de l'application Pokédex et mise en place de la CI

Cette cinquième semaine de stage a été consacrée à la finalisation de l'application Pokédex full-stack, avec pour objectifs l'affichage d'une liste de Pokémon stockés en base de données, ainsi que la possibilité d'en ajouter de nouveaux avec leur image associée.

J'ai débuté par la conception de la base de données, en optant pour MariaDB plutôt que MySQL: un choix motivé par une meilleure compatibilité avec Docker et par la présence native d'un script de healthcheck, permettant de vérifier que le service est réellement prêt à accepter des connexions avant le démarrage de l'API. J'ai mis en place deux volumes distincts : un volume nommé pour la persistance des données, et un volume en Bind Mount permettant l'exécution automatique des scripts SQL d'initialisation au premier démarrage du container.

Voir Annexe 6.1

J'ai ensuite développé le backend avec FastAPI, en connectant l'API à la base de données via le module mysql-connector, avant de conteneuriser l'ensemble via un Dockerfile basé sur l'image officielle python:3.13-slim.

La troisième journée a été consacrée à la sécurisation du projet : mise en place d'un fichier .env pour externaliser les identifiants sensibles de la base de données, création d'un .gitignore pour exclure les fichiers sensibles du suivi Git, ainsi qu'un fichier .env.example servant de modèle. J'ai également créé un Makefile centralisant les commandes Docker Compose courantes.

J'ai ensuite développé le frontend de l'application (HTML, CSS, JavaScript vanilla), directement servi par FastAPI via StaticFiles, avant de créer le repository GitHub du projet et un README détaillant son fonctionnement.

Voir Annexe 6.2

Cette semaine s'est conclue par la mise en place d'un pipeline CI via GitHub Actions, composée de deux jobs : le premier exécutant les tests automatisés (pytest) avec un service MariaDB simulé, le second vérifiant la bonne

construction de l'image Docker. Le projet a été présenté et validé par mon tuteur de stage en fin de semaine.

Voir Annexe 6.3

2.1.6 Mission 6 - Test de déploiement avec Coolify sur VirtualBox

Cette sixième semaine de stage, moins chargée que les précédentes, a été consacrée au pré-déploiement de l'application Pokédex conçue la semaine précédente, à travers l'utilisation de **Coolify**, un outil PaaS (Platform as a Service) open source auto-hébergé permettant de déployer et gérer des applications conteneurisées via une interface web, à la manière de services comme Heroku ou Railway.

J'ai débuté par la mise en place d'une machine virtuelle **VirtualBox** sous Debian, sur laquelle j'ai installé Docker et Git, prérequis indispensables au fonctionnement de Coolify, avant d'installer et de configurer l'outil lui-même.

Voir Annexe 7.1

J'ai ensuite connecté mon repository GitHub à Coolify afin de déployer automatiquement l'application via Docker Compose, en configurant les variables d'environnement nécessaires directement dans l'interface. Cette étape a révélé une première difficulté : Coolify recherchait par défaut un fichier docker-compose.yaml, alors que le mien portait l'extension .yml: un simple ajustement de configuration a permis de résoudre ce blocage.

Voir Annexe 7.2 et 7.3

J'ai ensuite rencontré un conflit de ports, Coolify et mon application FastAPI utilisant tous deux le port 8000 par défaut. J'ai résolu ce problème en

modifiant le port de mon application (Dockerfile, docker-compose.yml et script.js) vers le port 8081.

Voir Annexe 7.4

Enfin, j'ai été confronté à un problème plus complexe : la base de données restait vide malgré un déploiement réussi. L'investigation a révélé que Coolify avait créé des dossiers vides à la place des fichiers SQL d'initialisation attendus par Docker en bind mount: une limitation connue de l'outil. J'ai contourné ce problème en important manuellement les fichiers SQL directement dans le container MariaDB, rendant finalement l'application pleinement fonctionnelle.

Cette semaine m'a permis de comprendre concrètement les enjeux du déploiement en production, notamment la gestion des conflits de ports, des variables d'environnement, et les limitations propres aux outils PaaS comparés à un serveur classique offrant un contrôle total.

Voir Annexe 7.5

2.2 Mission 7 - Migration vers une infrastructure cloud AWS

La septième mission de ce stage, répartie sur les deux dernières semaines, constitue le point culminant de mon parcours : la migration complète de l'application Pokédex vers une infrastructure cloud professionnelle sur Amazon Web Services. Après avoir consolidé mes compétences en développement et en conteneurisation lors des missions précédentes, cette phase m'a permis de découvrir concrètement les enjeux d'un déploiement en environnement cloud: architecture réseau, sécurité, monitoring et haute disponibilité: bien au-delà de ce qu'un simple hébergement local ou un VPS classique aurait pu offrir.

2.2.1 - Semaine 7 - Déploiement initial sur AWS

Cette septième semaine a marqué le début de la mission la plus ambitieuse de mon stage : migrer l'application Pokédex, jusque-là testée en local et sur VirtualBox avec Coolify, vers une véritable infrastructure cloud professionnelle sur **Amazon Web Services**. Contrairement aux tentatives précédentes limitées par les contraintes d'un environnement virtualisé local, cette migration m'a permis de découvrir un écosystème complet de services managés, pensés pour la production.

Voir Annexe 1

A. Sécurisation du compte et mise en place de l'infrastructure de calcul

La première étape a consisté à sécuriser mon compte AWS avant toute manipulation technique. J'ai créé un utilisateur **IAM** dédié (plutôt que d'utiliser en permanence le compte root, une pratique fortement déconseillée en matière de sécurité), en lui attribuant la policy **AdministratorAccess**. J'ai activé l'authentification à deux facteurs (**MFA**) sur le compte root, et configuré des alertes de facturation (Budget AWS) à deux seuils distincts, un déclenchement immédiat au premier centime dépensé et un second à 5€, afin de garder un contrôle total sur les coûts tout au long du projet.

Voir Annexe 8.1

J'ai ensuite lancé mon instance EC2, en choisissant initialement un type t3.micro pour rester dans les limites du Free Tier. Ce choix s'est cependant révélé insuffisant : l'instance ne disposait que d'1 Go de RAM, quantité trop faible pour faire tourner Coolify de manière stable, provoquant des plantages répétés lors de l'installation. J'ai donc migré vers une instance t3.small (2 Go

de RAM), sortant ainsi du Free Tier mais garantissant un fonctionnement fiable de l'environnement de déploiement.

J'ai configuré les **Security Groups** de l'instance en n'ouvrant que les ports strictement nécessaires à son fonctionnement : le port 22 pour l'accès SSH, les ports 80 et 443 pour le trafic HTTP/HTTPS, le port 8000 pour l'interface d'administration Coolify, et le port 8081 pour exposer l'API. Chaque port ouvert représentant une surface d'attaque potentielle, j'ai veillé à ne jamais ouvrir plus que nécessaire. J'ai également associé une **Elastic IP** à l'instance, garantissant une adresse IP publique fixe qui ne change pas au redémarrage: contrairement à l'IP publique par défaut attribuée par AWS.

Voir Annexe 8.2

Une fois l'instance opérationnelle, j'ai installé Coolify via son script d'installation officiel, l'outil de déploiement continu que j'avais déjà expérimenté la semaine précédente sur ma machine virtuelle VirtualBox.

Voir Annexe 8.3

B. Mise en place d'une base de données managée avec Amazon RDS

Plutôt que de reconduire l'approche des semaines précédentes, où MariaDB tournait dans un simple container Docker aux côtés de l'application, j'ai choisi d'utiliser **Amazon RDS** (Relational Database Service), un service de base de données entièrement managé par AWS. Ce choix répondait à un double objectif : découvrir un service AWS fondamental largement utilisé en entreprise, et me rapprocher d'une architecture de production réelle où l'on cherche généralement à isoler la couche de données de la couche applicative.

J'ai lancé une instance **RDS MariaDB** en version db.t3.micro (compatible Free Tier), en la configurant grâce à l'option de connexion automatique à mon instance EC2 proposée par AWS lors de la création. Cette fonctionnalité configure automatiquement le réseau et les règles de sécurité nécessaires entre les deux services. J'ai ensuite placé cette base dans un **subnet privé**, distinct du subnet public hébergeant mon EC2, et configuré son Security Group pour n'autoriser les connexions sur le port 3306 (MySQL/MariaDB) qu'en provenance de mon instance EC2 exclusivement : rendant la base de données totalement inaccessible depuis internet, même en cas de connaissance de son endpoint.

Voir Annexe 8.4

J'ai ensuite testé la connexion depuis mon instance EC2 en SSH, avant d'importer le schéma SQL ainsi que les données Pokémon déjà développés durant les semaines précédentes, en veillant à respecter l'ordre d'exécution des fichiers (schéma avant données) pour éviter les erreurs déjà rencontrées lors des semaines de développement local.

Voir Annexe 8.5

C. Déploiement de l'application via Coolify

Une fois l'infrastructure de base en place, j'ai connecté mon repository GitHub public à Coolify, en sélectionnant le mode de déploiement adapté à mon fichier **docker-compose.yml**. J'ai configuré les variables d'environnement de connexion à la base de données, en remplaçant les valeurs pointant auparavant vers un container local par l'endpoint réel de mon instance RDS.

Voir Annexe 8.6

Le premier déploiement a révélé un problème inattendu : bien que les deux containers démarrent correctement, l'accès à l'API retournait une erreur de connectivité. L'investigation a révélé que le port 8081 de l'application n'était pas encore ouvert dans les Security Groups de l'EC2 : une omission corrigée en ajoutant la règle manquante.

J'ai également profité de cette étape pour simplifier l'architecture de déploiement : le frontend, initialement pensé comme un service séparé nécessitant potentiellement un serveur Nginx dédié, a finalement été directement intégré à l'API FastAPI via **StaticFiles**, FastAPI étant déjà capable de servir des fichiers statiques (HTML, CSS, JavaScript) sans nécessiter de composant supplémentaire: une solution plus légère et plus simple à maintenir.

Voir Annexe 8.7

D. Exploration du réseau AWS et mise en place du stockage S3

Pour clôturer cette semaine, j'ai pris le temps d'explorer en détail le fonctionnement du VPC (Virtual Private Cloud) créé automatiquement par AWS. J'ai identifié les différents subnets composant ce réseau, répartis entre subnets publics (accueillant mon instance EC2) et subnets privés (accueillant mon instance RDS), ainsi que l'Internet Gateway, service faisant office de passerelle entre mon réseau privé virtuel et internet: sans elle, aucune ressource de mon VPC ne serait accessible ni ne pourrait accéder à internet.

Voir Annexe 8.8

Cette exploration m'a permis de réaliser que, sans intervention manuelle particulière de ma part, j'avais déjà mis en place une architecture réseau à deux niveaux, avec séparation claire entre les composants exposés publiquement et ceux devant rester isolés: un principe fondamental de sécurité réseau que j'avais jusque-là uniquement étudié en théorie.

Enfin, j'ai créé mon premier bucket S3 (Simple Storage Service), destiné à héberger les images des Pokémon de manière indépendante du serveur applicatif. J'ai configuré une politique de bucket autorisant la lecture publique, puis uploadé manuellement l'ensemble des images existantes via AWS CLI. Cette approche garantit la persistance des assets même en cas de suppression ou de recréation complète de l'instance EC2, contrairement à un stockage local qui aurait été perdu dans un tel scénario.

Voir Annexe 8.9

2.2.2 Semaine 8 - Sécurisation et amélioration de l'infrastructure

Cette huitième et dernière semaine de stage a été consacrée à l'amélioration et à la sécurisation de l'infrastructure AWS mise en place la semaine précédente. Après avoir déployé une première version fonctionnelle de l'application, j'ai souhaité aller plus loin en renforçant la sécurité des communications, en mettant en place un monitoring proactif, et en automatisant certaines tâches jusque-là manuelles.

A. Sécurisation des communications avec HTTPS

La première amélioration a consisté à sécuriser les échanges entre le frontend et l'API en passant du protocole **HTTP**, non chiffré, au protocole **HTTPS**. J'ai généré un nom de domaine gratuit via **sslip.io**, directement intégré à Coolify, qui s'est chargé de générer et renouveler automatiquement un certificat SSL via **Let's Encrypt**.

Cette configuration a révélé un problème inattendu de stockage : le déploiement échouait avec une erreur indiquant que le disque était plein. En investiguant, j'ai constaté que le volume **EBS** de mon instance, initialement dimensionné à 8 Go, était saturé à 94%, principalement à cause de l'accumulation d'images Docker successives générées par les redéploiements répétés effectués tout au long des deux semaines. J'ai résolu ce problème en étendant le volume EBS à 20 Go directement depuis la console AWS, puis en étendant la partition et le système de fichiers correspondants avec les commandes **growpart** et **resize2fs**, sans perte de données.

Voir Annexe 9.1

Une fois le disque étendu, le déploiement s'est déroulé normalement et le certificat SSL a été généré avec succès. Cependant, un second problème est apparu : le navigateur bloquait silencieusement les requêtes vers l'API, car celle-ci était encore appelée en HTTP depuis une page désormais servie en HTTPS. Ce comportement, appelé **Mixed Content**, est un mécanisme de sécurité des navigateurs modernes interdisant le chargement de ressources non sécurisées depuis une page sécurisée. J'ai résolu ce problème en mettant à jour les fichiers config.js et script.js pour qu'ils utilisent l'URL HTTPS.
Voir Annexe 9.2

B. Mise en place du monitoring avec CloudWatch

Afin de mieux sécuriser mon infrastructure et d'anticiper toute surconsommation de ressources pouvant entraîner une hausse de la facturation AWS, j'ai mis en place un système de monitoring via Amazon CloudWatch. J'ai configuré au total 6 alarmes, réparties entre mon instance EC2 et ma base de données RDS, chacune associée à un seuil de déclenchement défini en fonction du comportement normal observé sur chaque métrique.

Concernant l'EC2, j'ai configuré une alarme sur la métrique CPUUtilization avec un seuil à 80% (l'usage normal se situant autour de 9%), une alarme sur la métrique NetworkIn avec un seuil à 2,5 Mo permettant de détecter un trafic entrant anormal, et une alarme sur la métrique StatusCheckFailed se déclenchant dès la détection d'un dysfonctionnement système. Les deux premières déclenchent un arrêt automatique de l'instance pour éviter toute surconsommation prolongée ou toute atteinte à la disponibilité (attaque de type DDoS notamment), tandis que la troisième déclenche un redémarrage automatique, plus adapté pour tenter de résoudre un problème système sans aggraver la situation.

Voir Annexe 9.3

Concernant RDS, j'ai configuré une alarme CPUUtilization avec un seuil à 75%, une alarme FreeStorageSpace se déclenchant sous 5 Go d'espace disque restant (en écho direct à la problématique de disque rencontrée sur l'EC2

quelques jours plus tôt), et une alarme DatabaseConnections avec un seuil de 10 connexions simultanées. Contrairement aux alarmes EC2, aucune action automatique n'a été associée à ces alarmes RDS, une intervention automatique sur une base de données pouvant entraîner un risque de perte ou de corruption de données.

Voir Annexe 9.4

Pour recevoir ces alertes, j'ai créé des topics SNS (Simple Notification Service) auxquels je me suis abonné par email, fonctionnant sur le principe d'une liste de diffusion. Enfin, j'ai centralisé l'ensemble de ces métriques au sein d'un dashboard CloudWatch, avec un widget dédié à chaque métrique pour conserver une lisibilité optimale malgré la diversité des unités mesurées (pourcentages, octets, nombre de connexions).

Voir Annexe 9.5

C. Automatisation du stockage avec Amazon S3

J'ai ensuite automatisé le processus d'upload des images vers S3, jusque-là réalisé manuellement. Pour cela, j'ai créé un utilisateur IAM dédié, distinct de mon compte administrateur, avec une policy limitée aux seules actions GetObject et PutObject sur mon bucket spécifique, respectant ainsi le principe du moindre privilège.

J'ai intégré le SDK boto3 à mon backend FastAPI, puis modifié la logique d'enregistrement des images : plutôt que d'être sauvegardées localement sur le disque de l'instance EC2, elles sont désormais directement envoyées vers le bucket S3, l'URL générée étant automatiquement stockée dans la base de données.

Voir Annexe 9.6

Cette implémentation n'a pas été sans difficultés. Mon pipeline CI a d'abord échoué avec une erreur ModuleNotFoundError: boto3, la dépendance n'ayant pas été ajoutée à l'étape d'installation des paquets dans mon workflow GitHub Actions. Une fois corrigée, une seconde erreur est survenue en production : NameError: s3_client is not defined, provenant d'un oubli d'initialisation du client S3 en début de fichier.

Voir Annexe 9.7

Par ailleurs, en testant l'application sur mon téléphone, j'ai constaté qu'elle ne parvenait pas à charger les données, contrairement à un fonctionnement normal sur ordinateur. L'investigation a révélé que le fichier script.js, n'étant pas versionné sur GitHub, avait été réécrasé lors d'un précédent redéploiement avec une ancienne version contenant une adresse IP en HTTP codée en dur. Les navigateurs mobiles appliquant une politique de sécurité Mixed Content plus stricte que les navigateurs desktop, ce problème n'était pas apparu lors des tests précédents sur PC. La correction du fallback dans **script.js** a résolu ce problème.
Voir Annexe 9.8

D. Documentation technique et finalisation

Les derniers jours de cette semaine ont été consacrés à la réalisation d'un **MCD/MLD** de ma base de données, permettant d'expliquer certains choix de conception, notamment le stockage des types de Pokémon en JSON plutôt qu'au sein d'une table de liaison dédiée, ainsi que l'auto-référence utilisée pour représenter les chaînes d'évolution.

Voir Annexe 9.9

J'ai également réalisé un schéma applicatif et mis à jour mon schéma d'architecture AWS pour y intégrer CloudWatch, SNS, ainsi que l'utilisateur IAM dédié à S3, et rédigé l'ensemble de la documentation technique du projet : documentation de l'API, guide de déploiement AWS, et mise à jour complète du README du repository GitHub.

Voir Annexe 9.10 et 9.11

Cette dernière semaine s'est conclue par la clôture administrative du stage, avec une réunion finale avec mon tuteur de stage permettant de dresser un bilan global de ces huit semaines.

3. Compétences techniques acquises

Ce stage de huit semaines m'a permis de développer un ensemble de compétences techniques variées, allant du développement applicatif à l'administration d'infrastructures cloud, en passant par la sécurité et l'automatisation des déploiements.

1. Développement et programmation

- Conception et développement d'une **API REST** avec le framework **FastAPI** (Python)
- Manipulation de bases de données relationnelles via le module **mysql-connector-python**
- Intégration d'un **frontend** en HTML, CSS et JavaScript vanilla (développement assisté par IA), avec compréhension du mécanisme de communication asynchrone avec l'API (fetch, async/await)
- Utilisation du SDK **boto3** pour interagir avec les services AWS depuis un environnement Python

2. Conteneurisation et orchestration

- Création et gestion d'images **Docker** (Dockerfile, bonnes pratiques de construction d'image)
- Orchestration de services multi-conteneurs avec **Docker Compose**
- Gestion des volumes Docker (persistance des données, bind mounts)
- Automatisation de commandes courantes via un **Makefile**

3. Versioning et intégration continue

- Maîtrise des outils Git et GitHub (branches, merge, gestion de conflits, authentification SSH)
- Conception d'un pipeline CI (Continuous Integration) avec GitHub Actions : tests automatisés (pytest), vérification de la construction d'image Docker

- Compréhension du fonctionnement d'un pipeline CD (Continuous Deployment) via Coolify

4. Infrastructure et services cloud AWS

- Gestion des accès et de la sécurité via IAM (utilisateurs dédiés, policies personnalisées, principe du moindre privilège)
- Déploiement et administration d'instances EC2 (Ubuntu, dimensionnement, Elastic IP)
- Mise en place d'une base de données managée avec RDS (MariaDB)
- Configuration d'un réseau VPC (subnets publics/privés, Internet Gateway, Security Groups)
- Stockage d'objets avec Amazon S3 (bucket, politique de lecture publique, upload automatisé)
- Supervision et alerting avec CloudWatch (métriques, alarmes, dashboards) et SNS (notifications)
- Extension de volumes de stockage EBS et gestion des partitions Linux (growpart, resize2fs)

5. Sécurité

- Sécurisation des communications via HTTPS (certificats SSL avec Let's Encrypt)
- Isolation réseau selon le principe de défense en profondeur (subnets privés, Security Groups restrictifs)
- Application du principe du moindre privilège via des utilisateurs IAM dédiés à des tâches spécifiques
- Compréhension et résolution de problématiques de sécurité navigateur (Mixed Content)

6. Administration système Linux

- Administration d'un serveur **Ubuntu** via SSH **(authentification par clé)**
- Gestion de l'espace disque et diagnostic de saturation **(df -h, docker system prune)**

- Manipulation de conteneurs en production **(logs, exec, inspection réseau Docker)**

7. Documentation technique

- Rédaction de documentation technique structurée en Markdown (documentation API, guide de déploiement)
- Réalisation de schémas d'architecture avec Draw.io (infrastructure cloud, pipeline CI/CD, modélisation de données)
- Modélisation de base de données (MCD/MLD)

4. Compétences professionnelles développées

Au-delà des compétences techniques, ce stage m'a permis de développer des compétences professionnelles transversales, essentielles à mon intégration future en entreprise.

1. **Communication et collaboration**
    - Communication régulière avec mon équipe via WhatsApp et Gmail pour les échanges quotidiens
    - Participation à des réunions de suivi en visioconférence via Google Meet, avec préparation et prise de notes structurées
    - Capacité à présenter et expliquer un travail technique à des professionnels expérimentés (démonstrations de mon projet à mon tuteur de stage)

2. **Autonomie et organisation**
    - Structuration autonome de mon apprentissage, avec prise de notes régulières facilitant la mémorisation et la progression
    - Gestion de mon temps sur des missions hebdomadaires avec des objectifs définis en début de semaine
    - Capacité à identifier mes propres lacunes techniques et à orienter ma veille en conséquence

3. **Résolution de problèmes (troubleshooting)**
    - Démarche méthodique de diagnostic face à des problèmes techniques complexes (analyse de logs, isolement des causes, tests progressifs)
    - Capacité à résoudre des problèmes de manière autonome, en combinant documentation officielle et recherche personnelle
    - Persévérance face à des blocages techniques (conflits de ports, saturation disque, problèmes de connectivité réseau)

4. Rigueur et bonnes pratiques professionnelles

- Application de bonnes pratiques de sécurité (gestion des accès, secrets, principe du moindre privilège) même dans un contexte de projet personnel
- Souci de la documentation et de la reproductibilité d'un projet (README, guide de déploiement, fichiers d'exemple)
- Vigilance sur la gestion des coûts dans un environnement cloud (surveillance de la facturation, alertes budgétaires)

5. Veille technologique et adaptabilité

- Capacité à apprendre rapidement de nouvelles technologies (AWS, CloudWatch, S3) en autonomie
- Utilisation raisonnée de l'intelligence artificielle comme outil d'apprentissage et d'assistance technique
- Adaptation face à des imprévus techniques nécessitant de revoir une approche initiale (changement de type d'instance EC2, restructuration de l'architecture de déploiement)

5. Bilan personnel

Ce stage de huit semaines, mon tout premier en entreprise, a été une expérience particulièrement enrichissante, tant sur le plan technique que personnel.

Ce que ce stage m'a apporté

J'ai pu mesurer concrètement l'écart entre les notions théoriques abordées en cours et leur mise en pratique réelle en entreprise. La mission de migration vers AWS, en particulier, m'a permis de comprendre qu'une infrastructure ne se résume jamais à "faire fonctionner une application": elle implique une réflexion constante sur la sécurité, les coûts, la supervision et la résilience face aux imprévus.

J'ai également pris conscience de l'importance de la documentation et de la reproductibilité d'un projet, des aspects que j'avais tendance à sous-estimer avant ce stage, mais qui se sont révélés indispensables à chaque étape, que ce soit pour reprendre mon propre travail après une interruption ou pour permettre à un tiers de comprendre et reproduire mon infrastructure.

Difficultés rencontrées

Les principales difficultés de ce stage ont été d'ordre technique : la résolution de problèmes complexes et parfois inattendus (saturation de disque, conflits réseau, incompatibilités entre navigateurs) a demandé beaucoup de patience et de méthode. J'ai également dû apprendre à mieux structurer mon temps de travail, notamment lors des premières semaines davantage théoriques, où l'absence de livrable concret rendait parfois difficile la mesure de ma propre progression.

Sur le plan professionnel, évoluer au sein d'une très petite structure (deux personnes, toutes deux impliquées par ailleurs dans d'autres activités professionnelles) a nécessité une bonne dose d'autonomie, les échanges avec mon tuteur de stage n'étant pas toujours quotidiens.

Ce que je ferais différemment

Si c'était à refaire, je consacrerais davantage de temps, dès le début du stage, à la mise en place d'une infrastructure en tant que code avec un outil comme **Terraform**, plutôt que de configurer manuellement chaque service AWS via la console web. Cela aurait permis une infrastructure plus facilement reproductible et versionnée, et m'aurait évité de perdre du temps sur des reconfigurations manuelles répétées.

Perspectives

Ce stage a confirmé mon intérêt pour les métiers du cloud, de la sécurité et du DevOps, un domaine que je souhaite approfondir durant ma deuxième année de BTS SIO, notamment à travers l'apprentissage de **Terraform** pour l'infrastructure as code, et de **Kubernetes** pour l'orchestration de conteneurs à plus grande échelle. Je souhaite également poursuivre ma préparation à la certification **AWS Certified Cloud Practitioner (CLF-C02)**, qui constituerait une première reconnaissance officielle de mes compétences dans ce domaine.

À l'issue de mon BTS, j'envisage de poursuivre mes études en alternance, idéalement sur un poste orienté **Cloud/DevOps**, ou sur un poste d'administrateur systèmes et réseaux avec une appétence pour le cloud AWS ou Azure: ce dernier étant un fournisseur très populaire en France que je souhaiterais également approfondir si le temps me le permet; afin de continuer à développer une expertise technique tout en découvrant de nouveaux environnements professionnels.

Conclusion

Ce stage de huit semaines chez Consultas Group aura été une expérience déterminante dans la construction de mon projet professionnel. Parti de bases théoriques en développement d'API et en conteneurisation, j'ai progressivement acquis les compétences nécessaires pour concevoir, développer et déployer une application complète sur une infrastructure cloud professionnelle.

La mission de migration vers Amazon Web Services, point culminant de ce stage, m'aura permis de découvrir concrètement les enjeux d'une infrastructure de production : sécurité réseau, gestion des accès, supervision proactive et maîtrise des coûts. Au-delà des compétences techniques acquises, ce stage m'aura surtout appris qu'une infrastructure fiable se construit autant par l'anticipation des risques que par la capacité à diagnostiquer et résoudre efficacement les problèmes rencontrés.

Cette première expérience en entreprise confirme mon souhait de m'orienter vers les métiers du cloud et du DevOps, et me donne les bases nécessaires pour poursuivre cet apprentissage tout au long de ma deuxième année de BTS SIO, puis dans le cadre d'une future alternance.

Glossaire

API (Application Programming Interface) : Interface permettant à deux applications de communiquer entre elles selon un ensemble de règles définies.

API REST : Style d'architecture pour la conception d'API, basé sur le protocole HTTP et l'utilisation de méthodes standardisées (**GET**, **POST**, **PUT**, **DELETE**).

AWS (Amazon Web Services) : Plateforme de services cloud proposée par Amazon, regroupant des centaines de services (calcul, stockage, bases de données, réseau...).

Boto3 : SDK officiel d'AWS pour le langage Python, permettant d'interagir avec l'ensemble des services AWS (S3, EC2, IAM...) directement depuis du code.

CI/CD (Continuous Integration / Continuous Deployment) : Ensemble de pratiques visant à automatiser les phases de test et de déploiement d'une application.

CloudWatch : Service AWS de supervision permettant de collecter des métriques et de configurer des alarmes sur les ressources cloud.

Conteneur (container) : Environnement isolé et léger permettant d'exécuter une application avec toutes ses dépendances, indépendamment du système hôte.

Coolify : Outil PaaS open source auto-hébergé permettant de déployer et gérer des applications conteneurisées via une interface web.

DevOps : Ensemble de pratiques et de culture visant à rapprocher les équipes de développement (Dev) et d'exploitation/infrastructure (Ops), en automatisant les cycles de développement, de test et de déploiement, afin de livrer des applications plus rapidement et plus fiablement.

Docker : Plateforme de conteneurisation permettant de créer, déployer et exécuter des applications dans des conteneurs.

Docker Compose : Outil permettant de définir et d'orchestrer plusieurs conteneurs Docker au sein d'une même application.

Dockerfile : Fichier texte contenant les instructions nécessaires à la construction d'une image Docker.

EBS (Elastic Block Store) : Service AWS de stockage par blocs, utilisé comme disque dur virtuel pour les instances EC2.

EC2 (Elastic Compute Cloud) : Service AWS permettant de louer des serveurs virtuels (instances) dans le cloud.

Elastic IP : Adresse IP publique fixe fournie par AWS, ne changeant pas au redémarrage d'une instance.

Endpoint : Point d'accès (URL) permettant d'interagir avec une API ou un service.

FastAPI : Framework Python moderne et performant destiné à la création d'API REST, s'appuyant sur le typage Python et générant automatiquement une documentation interactive.

Git : Système de gestion de versions décentralisé, permettant de suivre l'évolution du code source d'un projet.

GitHub : Plateforme d'hébergement de dépôts Git, facilitant la collaboration et le versioning de projets.

GitHub Actions : Service intégré à GitHub permettant d'automatiser des workflows, notamment des pipelines CI/CD.

HTTPS : Version sécurisée du protocole HTTP, chiffrant les communications entre un client et un serveur via un certificat SSL/TLS.

IaaS (Infrastructure as a Service) : Modèle cloud fournissant une infrastructure informatique de base (serveurs, stockage, réseau) à la demande, laissant à l'utilisateur la gestion du système d'exploitation et des applications.

IAM (Identity and Access Management) : Service AWS de gestion des identités et des accès, permettant de créer des utilisateurs et de définir leurs permissions.

Internet Gateway : Composant réseau AWS faisant office de passerelle entre un VPC et internet.

JSON (JavaScript Object Notation) : Format léger d'échange de données, structuré en paires clé-valeur.

Let's Encrypt : Autorité de certification gratuite délivrant des certificats SSL/TLS.

Makefile : Fichier de configuration permettant d'automatiser des commandes récurrentes via l'outil `make`.

MariaDB : Système de gestion de base de données relationnelle, dérivé de MySQL.

MCD/MLD (Modèle Conceptuel/Logique de Données) : Représentations schématiques d'une base de données, respectivement à un niveau conceptuel (abstrait) et logique (technique).

MFA (Multi-Factor Authentication) : Authentification à plusieurs facteurs, renforçant la sécurité d'un compte.

Mixed Content : Mécanisme de sécurité des navigateurs bloquant le chargement de ressources non sécurisées (HTTP) depuis une page sécurisée (HTTPS).

mysql-connector-python : Module Python officiel permettant d'établir une connexion et de communiquer avec une base de données MySQL ou MariaDB.

PaaS (Platform as a Service) : Modèle cloud fournissant une plateforme prête à l'emploi pour développer, déployer et gérer des applications, sans avoir à gérer l'infrastructure sous-jacente.

Pipeline : Enchaînement automatisé d'étapes successives (**tests, build, déploiement**) exécutées à chaque modification du code source.

Pattern : Modèle ou solution éprouvée répondant à un problème récurrent de conception logicielle ou d'architecture.

PR (Pull Request) : Demande de fusion d'une branche de code vers une autre sur une plateforme comme GitHub, généralement soumise à une revue avant validation.

Pydantic : Bibliothèque Python permettant la validation et la sérialisation de données à l'aide de modèles de classes, largement utilisée avec FastAPI.

Pytest : Framework de test pour le langage Python, utilisé pour l'écriture et l'exécution de tests automatisés.

RDS (Relational Database Service) : Service AWS de base de données relationnelle managée.

Repository (dépôt) : Espace de stockage d'un projet versionné avec Git, contenant l'historique complet des modifications du code source.

Root account : Compte administrateur principal d'un compte AWS, disposant de tous les droits sans restriction, dont l'usage quotidien est fortement déconseillé pour des raisons de sécurité.

SSH (Secure Shell) : Protocole permettant une connexion sécurisée et chiffrée à distance à un serveur.

S3 (Simple Storage Service) : Service AWS de stockage d'objets, utilisé notamment pour héberger des fichiers statiques.

SaaS (Software as a Service) : Modèle cloud fournissant une application directement utilisable par l'utilisateur final, sans installation ni gestion technique (ex : **Gmail, Google Docs**).

Scalabilité (scale) : Capacité d'un système à s'adapter à une augmentation de charge, en ajoutant des ressources (verticalement ou horizontalement) sans dégradation de performance.

Scope : Périmètre de permissions ou de visibilité accordé à un utilisateur, une clé d'accès ou une variable dans un contexte donné.

Security Group : Ensemble de règles de pare-feu associées à une ressource AWS, contrôlant le trafic réseau entrant et sortant.

SNS (Simple Notification Service) : Service AWS de notification, permettant l'envoi de messages (**email, SMS**...) vers des abonnés.

Subnet : Sous-réseau, portion d'un réseau plus large (VPC), pouvant être public ou privé.

Terraform : Outil d'Infrastructure as Code (IaC) permettant de décrire, provisionner et gérer une infrastructure cloud via des fichiers de configuration, garantissant reproductibilité et versioning.

Uvicorn : Serveur web ASGI utilisé pour exécuter des applications Python asynchrones comme FastAPI.

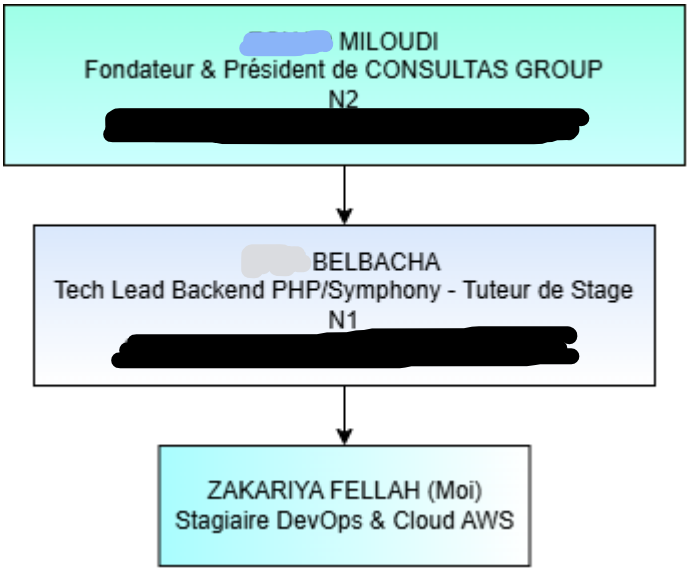
VPC (Virtual Private Cloud) : Réseau privé virtuel isolé au sein du cloud AWS, dans lequel sont déployées les ressources.

VPS (Virtual Private Server) : Serveur privé virtuel offrant un environnement dédié sur un serveur physique partagé.

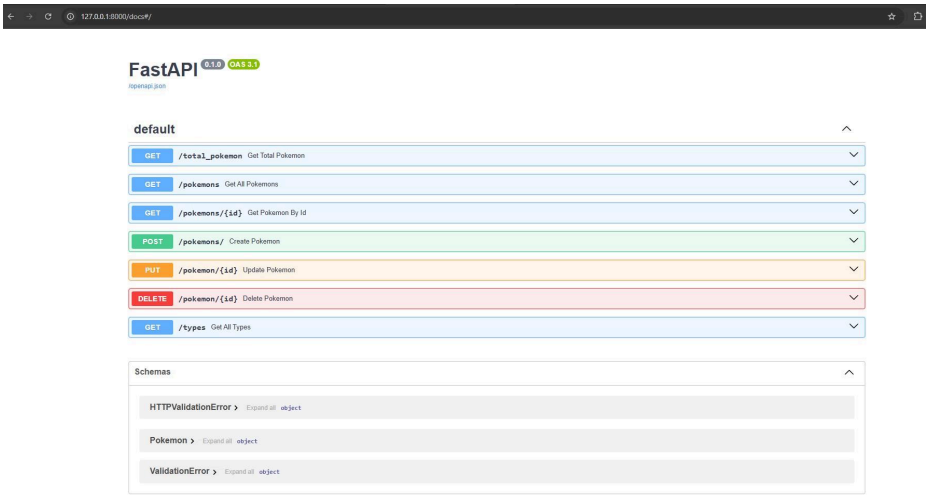
Kubernetes : Système open source d'orchestration de conteneurs, permettant d'automatiser le déploiement, la mise à l'échelle et la gestion d'applications conteneurisées à grande échelle.

Annexes

Annexe 1: *Repository Github Projet Pokédex* ***(Cliquez pour voir)***



Annexe 2 : Organigramme de l'équipe Consultas Group



Annexe 3.1 : Interface API (Swagger UI)

main.py

C: > Users > zaksa > Desktop > Stage > FastAPI > main.py

```python
from dataclasses import dataclass,asdict
from typing import Union
import json
from fastapi import FastAPI,Path,HTTPException

#===== Structure de données : Dictionnaire indexé par pokemon id =====#
with open("pokemons.json", "r") as f:
    pokemons_list = json.load(f)

list_pokemons = {k+1:v for k, v in enumerate(pokemons_list)}
#=====================================================================

@dataclass
class Pokemon() :
    id: int
    name: str
    types: list[str]
    total: int
    hp: int
    attack: int
    defense: int
    attack_special: int
    defense_special: int
    speed: int
    evolution_id: Union[int, None] = None
#=====================================================================#

app =FastAPI()

#=============================================GET=============================================#

@app.get("/total_pokemon") #Obtenir total de pokemons
def get_total_pokemon() -> dict:
    return {"total": len(list_pokemons)}

@app.get("/pokemons") #Obtenir liste de pokemons
def get_all_pokemons() -> list[Pokemon]:
    res = []
    for id in list_pokemons:
        res.append(Pokemon(**list_pokemons[id]))
    return res
```

```python
@app.get("/pokemons/{id}") #Obtenir un pokemon avec la condition qu'il soit dans la liste et avec un id qui est supérieur ou égal à 1
def get_pokemon_by_id(id:int = Path (ge=1)) -> Pokemon :
    if id not in list_pokemons:
        raise HTTPException(status_code=404, detail="Ce pokemon n'existe pas !")
    return Pokemon(**list_pokemons[id])

#=============================================POST=============================================#

@app.post("/pokemons/")
def create_pokemon(pokemon:Pokemon) -> Pokemon:
    if pokemon.id in list_pokemons :
        raise HTTPException(status_code=404,detail="Le pokemon {pokemon.id} existe déjà !")
    list_pokemons[pokemon.id] = asdict(pokemon)
    return pokemon

#=============================================PUT=============================================#

@app.put("/pokemon/{id}")
def update_pokemon(pokemon: Pokemon, id: int = Path(ge=1)) -> Pokemon:
    if id not in list_pokemons:
        raise HTTPException(status_code=404, detail="Ce pokemon n'existe pas !")

    list_pokemons[id] = asdict(pokemon)
    return pokemon

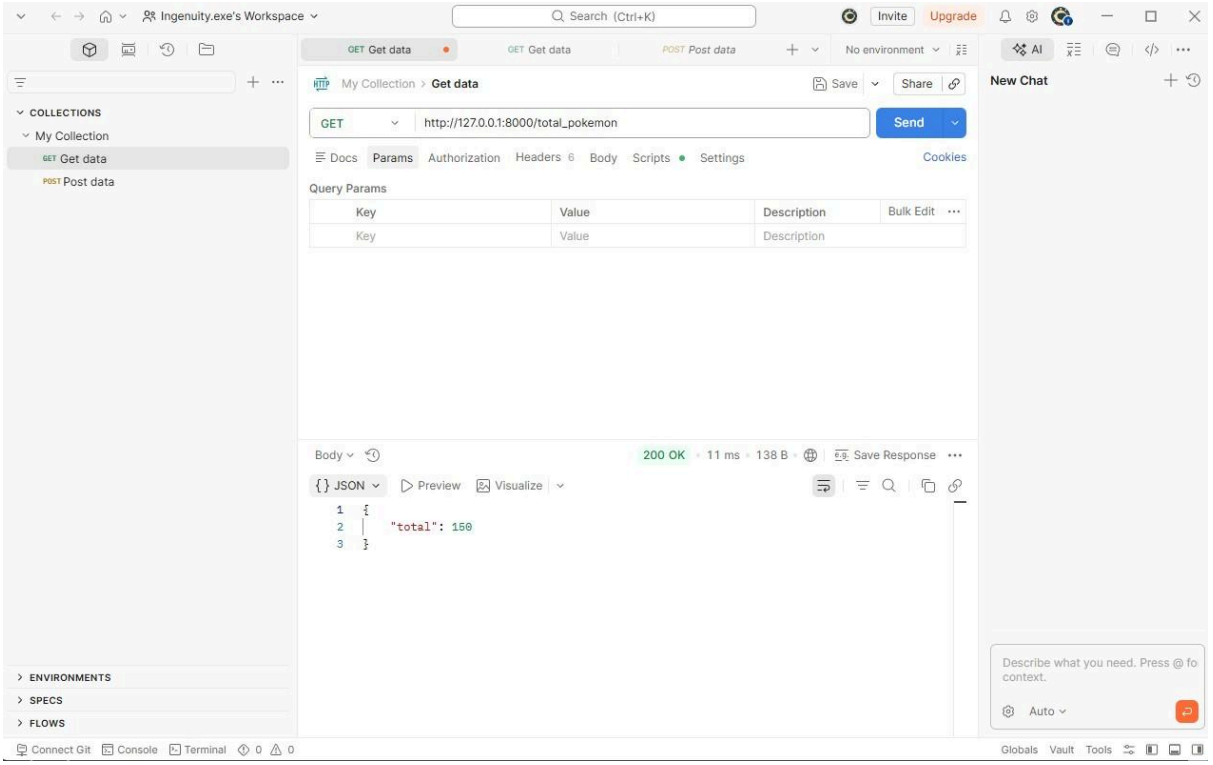
#=============================================DELETE=============================================#

@app.delete("/pokemon/{id}")
def delete_pokemon(id:int=Path(ge=1)) -> Pokemon:
    if id in list_pokemons:
        pokemon = Pokemon(**list_pokemons[id])
        del list_pokemons[id]
    return pokemon


#=============================================GET (Advanced Method)=============================================#

@app.get("/types")
def get_all_types()-> list[str]:
    types =[]
    for pokemon in pokemons_list:
        for type in pokemon["types"]:
            if type not in types:
                types.append(type)
    types.sort()
    return types
```

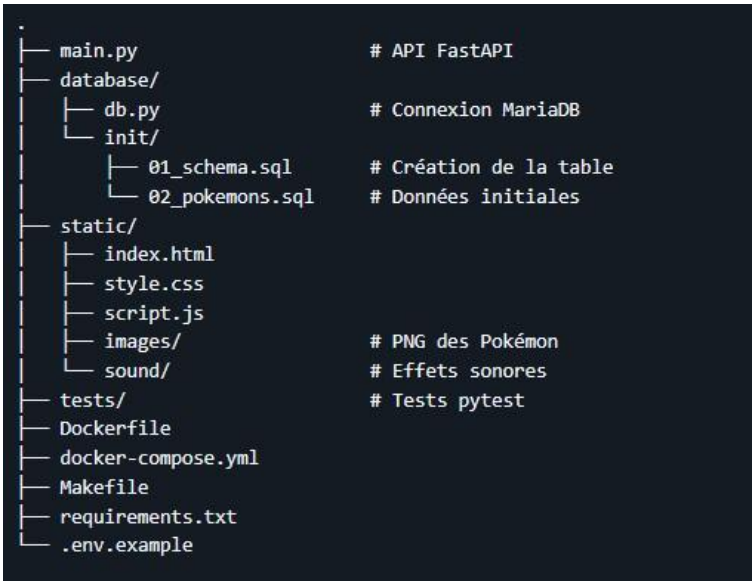
Annexe 3.2 : Code python (Framework FastAPI)



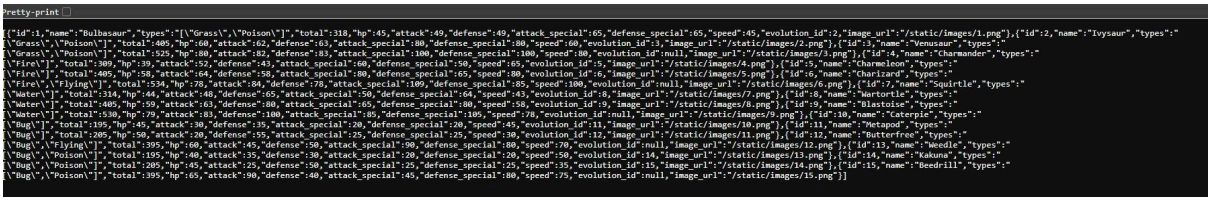
Annexe 4.1 : Requête HTTP PostMan (Test de l'API)



Annexe 4.2 : Makefile (alias Docker)



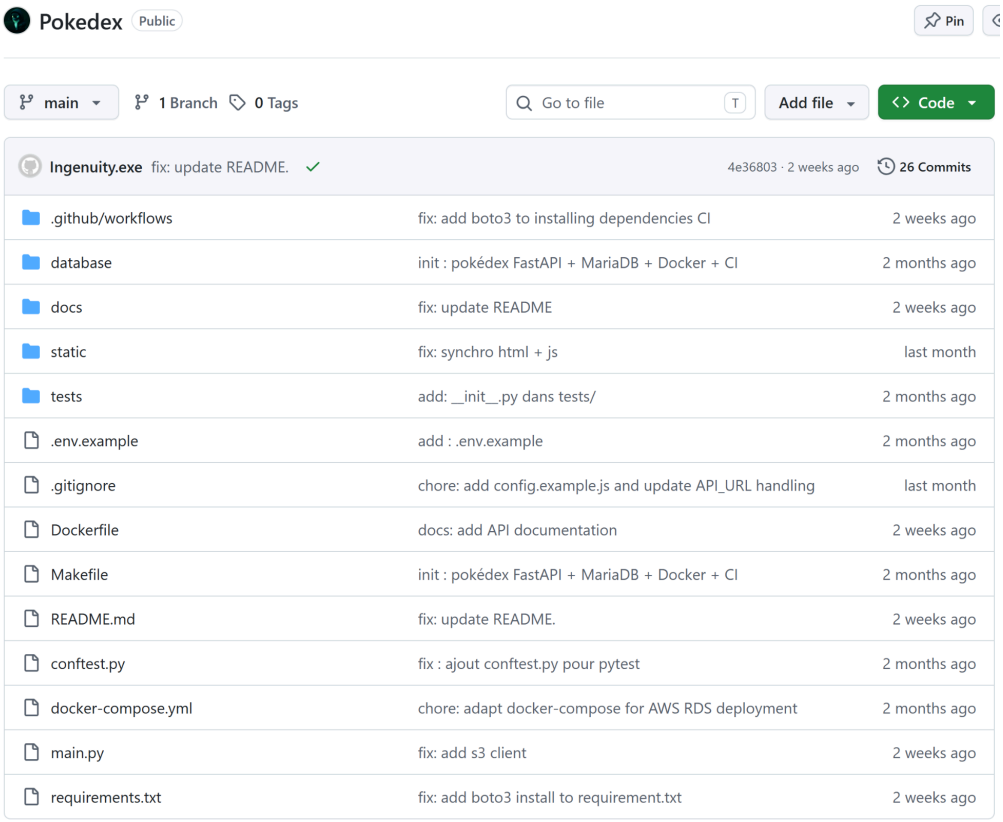
Annexe 5.1 : Structure des fichiers du projet



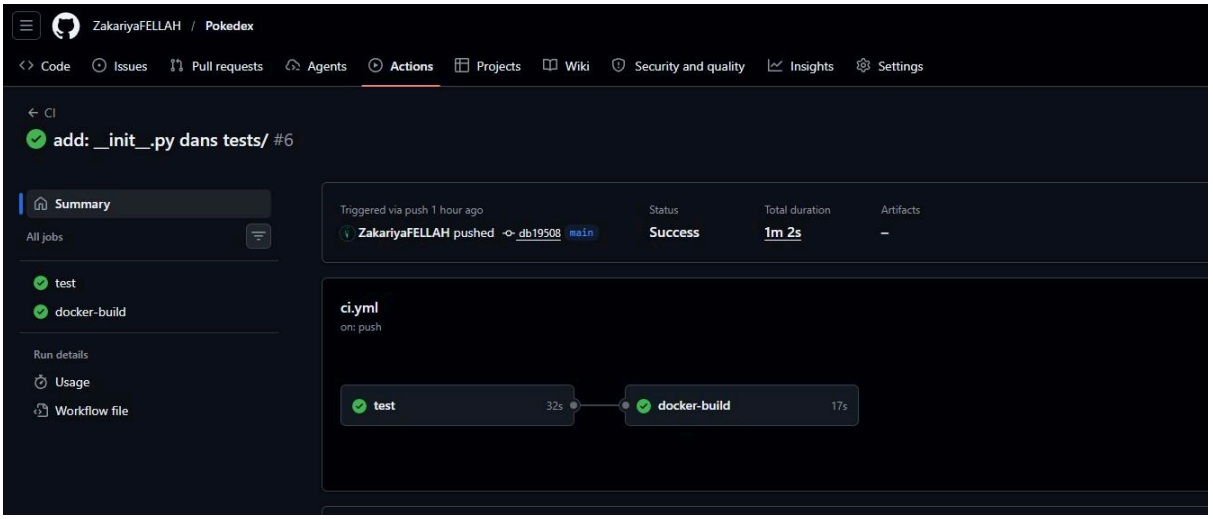
Annexe 5.2 : Test API en local

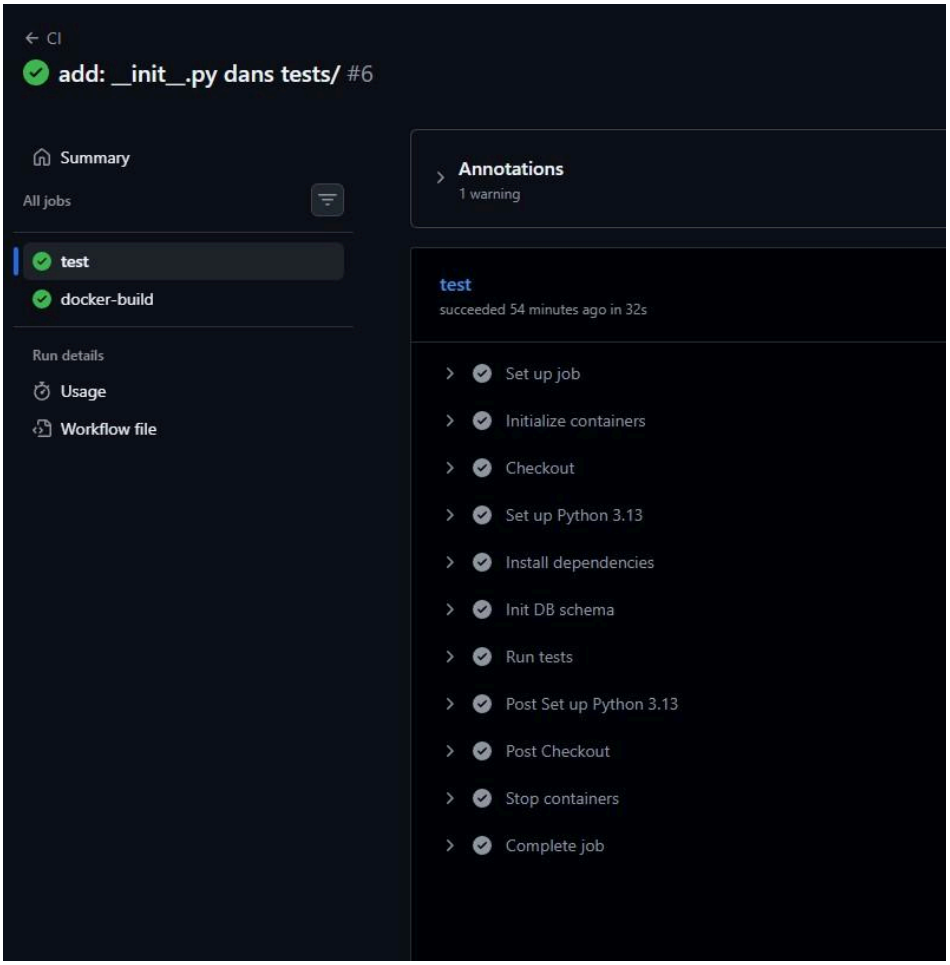
```sql
CREATE TABLE pokemon (
    id INT PRIMARY KEY,
    name VARCHAR(100),
    types JSON,
    total INT,
    hp INT,
    attack INT,
    defense INT,
    attack_special INT,
    defense_special INT,
    speed INT,
    evolution_id INT,
    image_url VARCHAR(255)
);
```

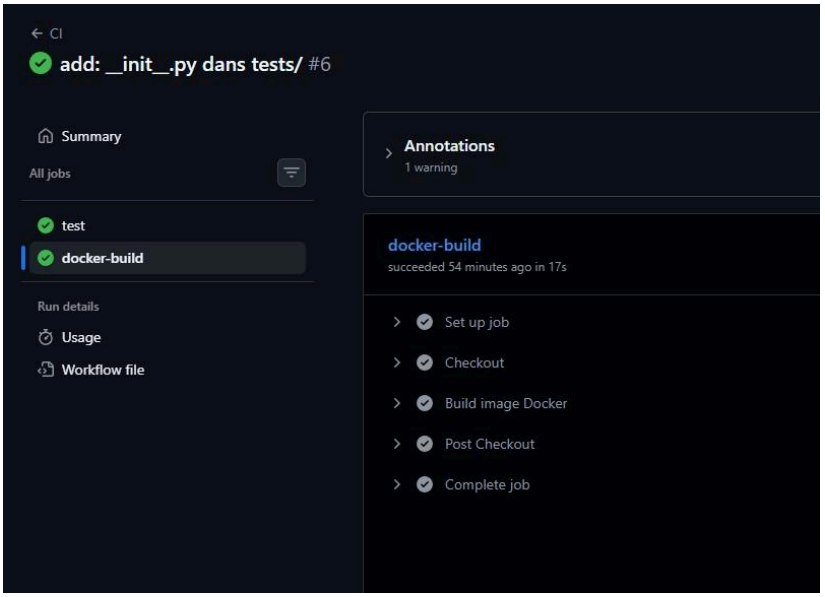
Annexe 6.1 : Structure base de données SQL



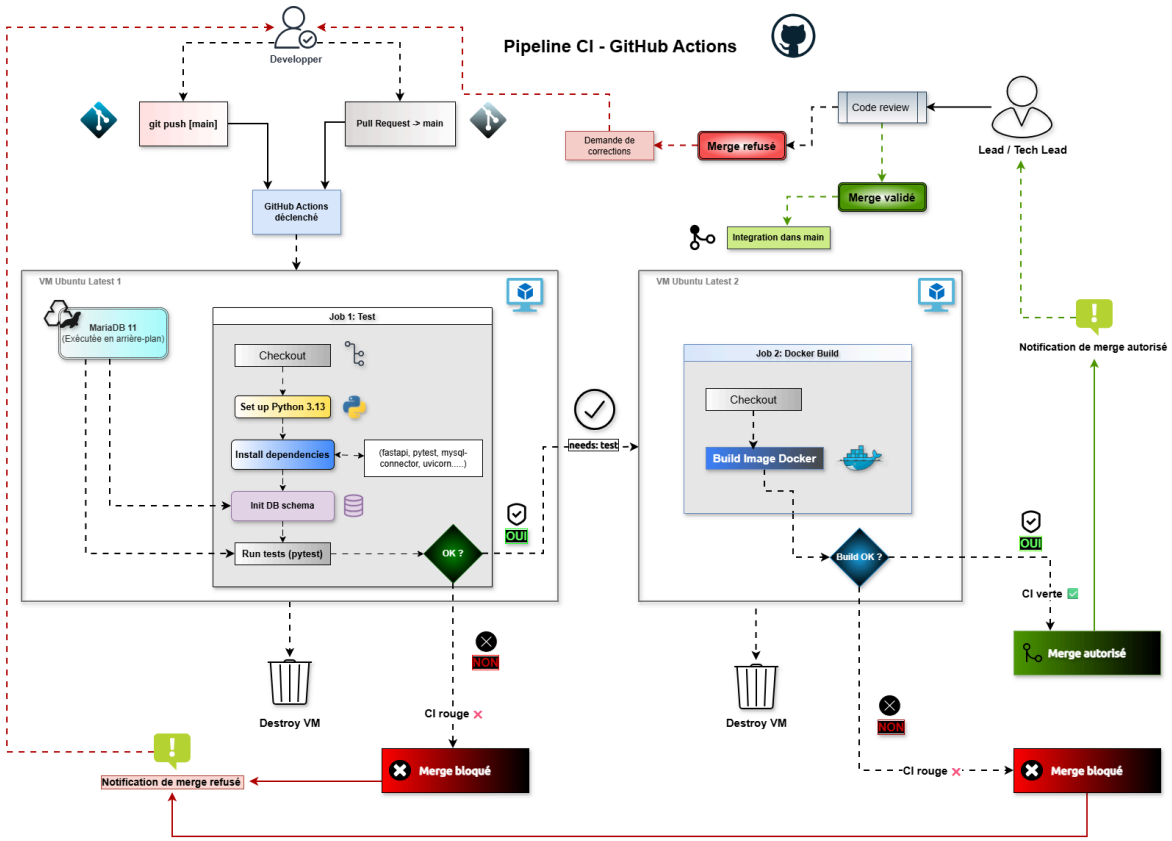
Annexe 6.2 : Repository Github (Pokédex)



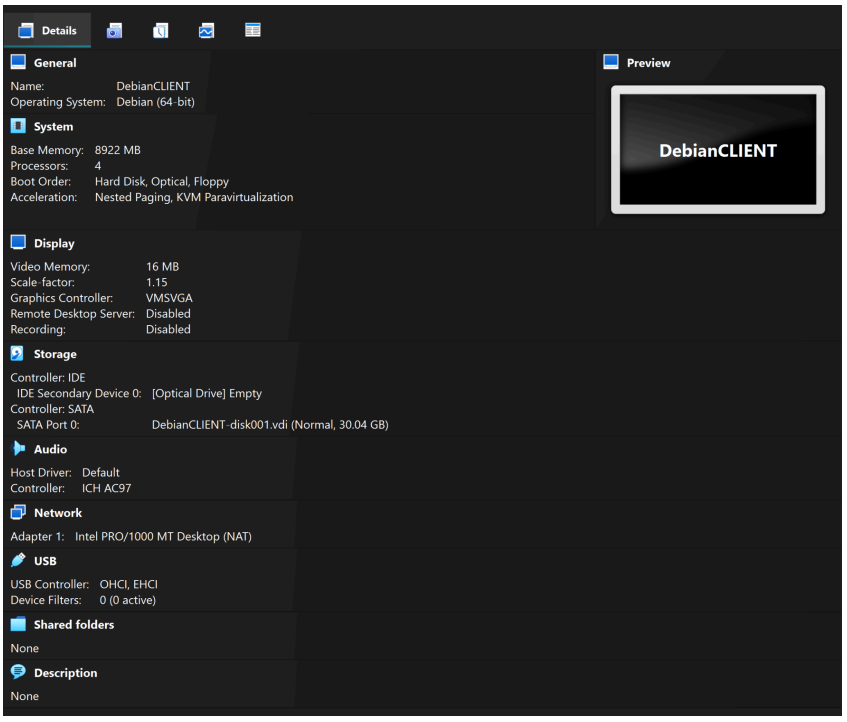




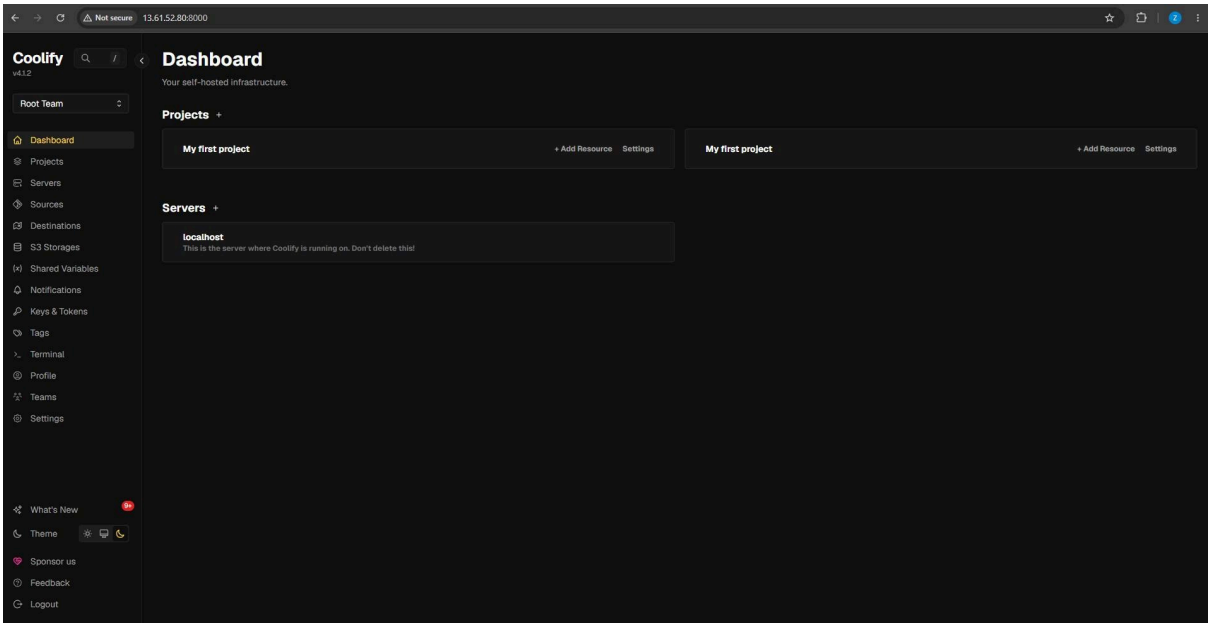
Annexe 6.3 : Pipeline CI - Github Action



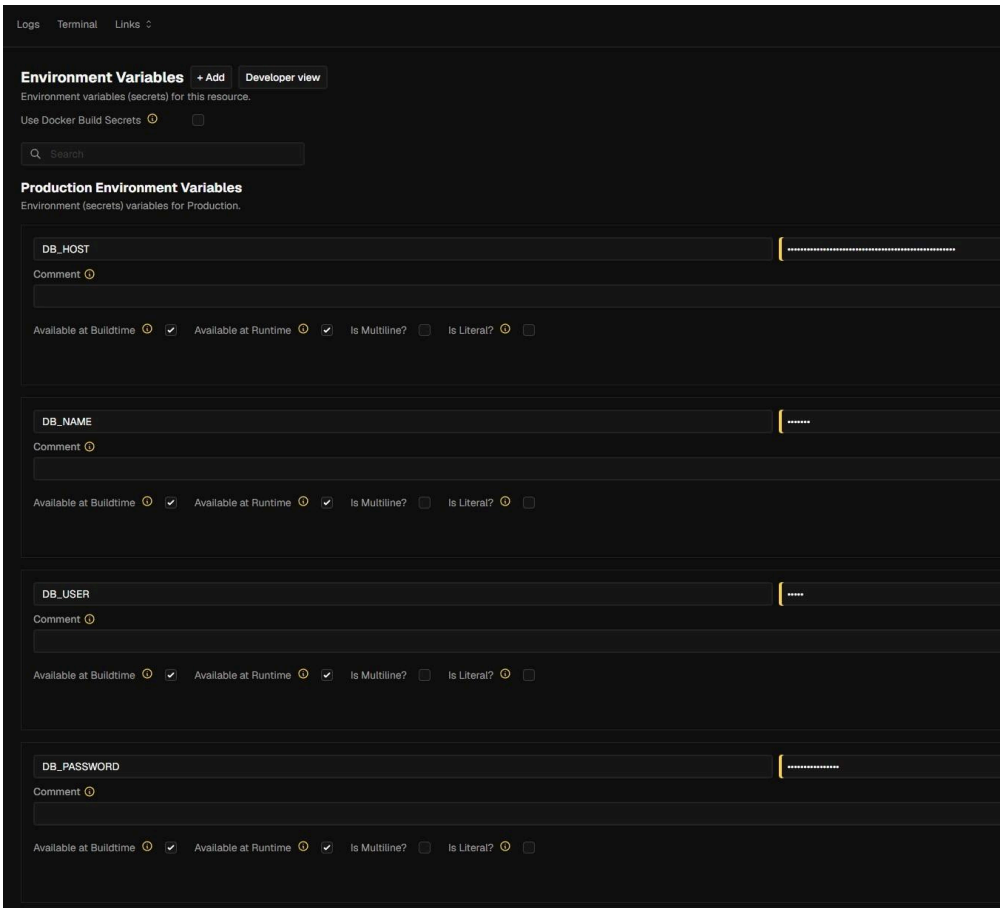
Annexe 6.4 : Pipeline CI GitHub Actions (schéma complet)



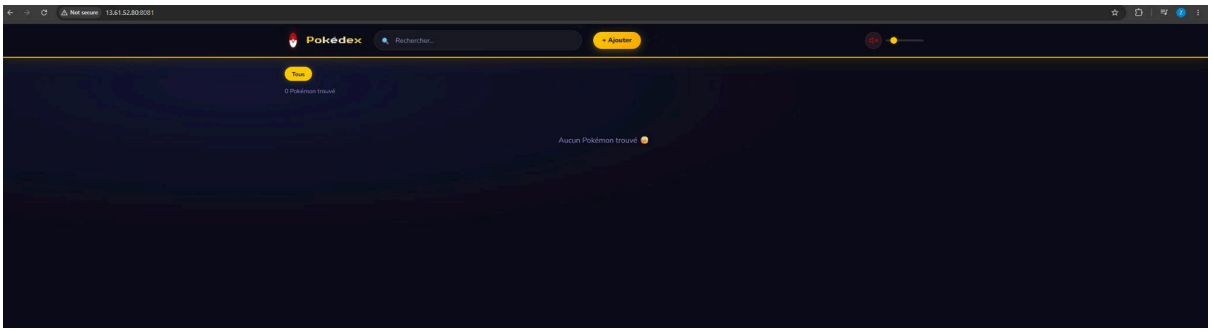
Annexe 7.1 : VM VirtualBox configurée (Debian)



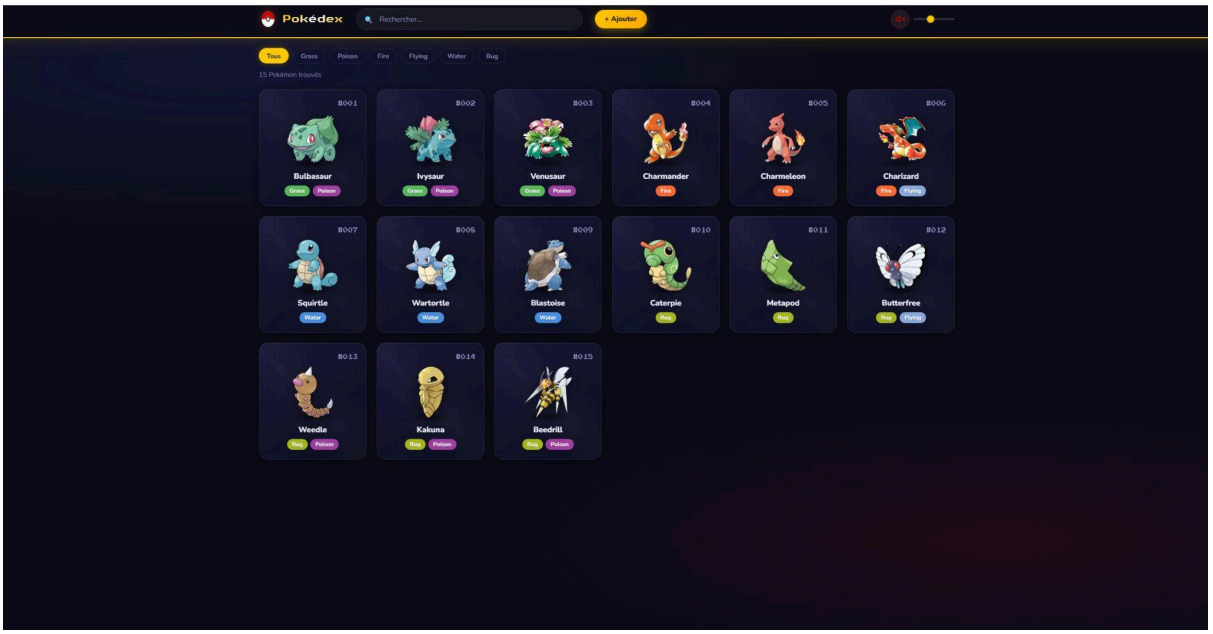
Annexe 7.2 : Interface Coolify



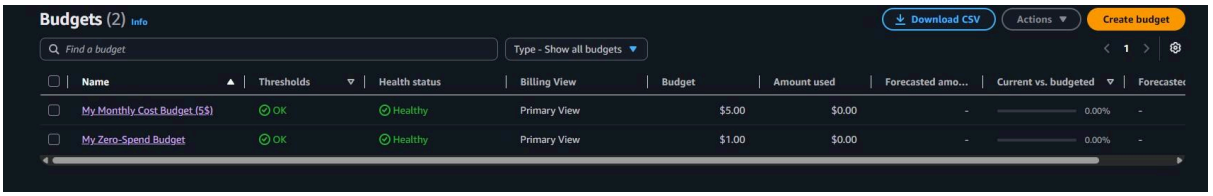
Annexe 7.3 : Variables d'environnement Coolify (test VirtualBox)

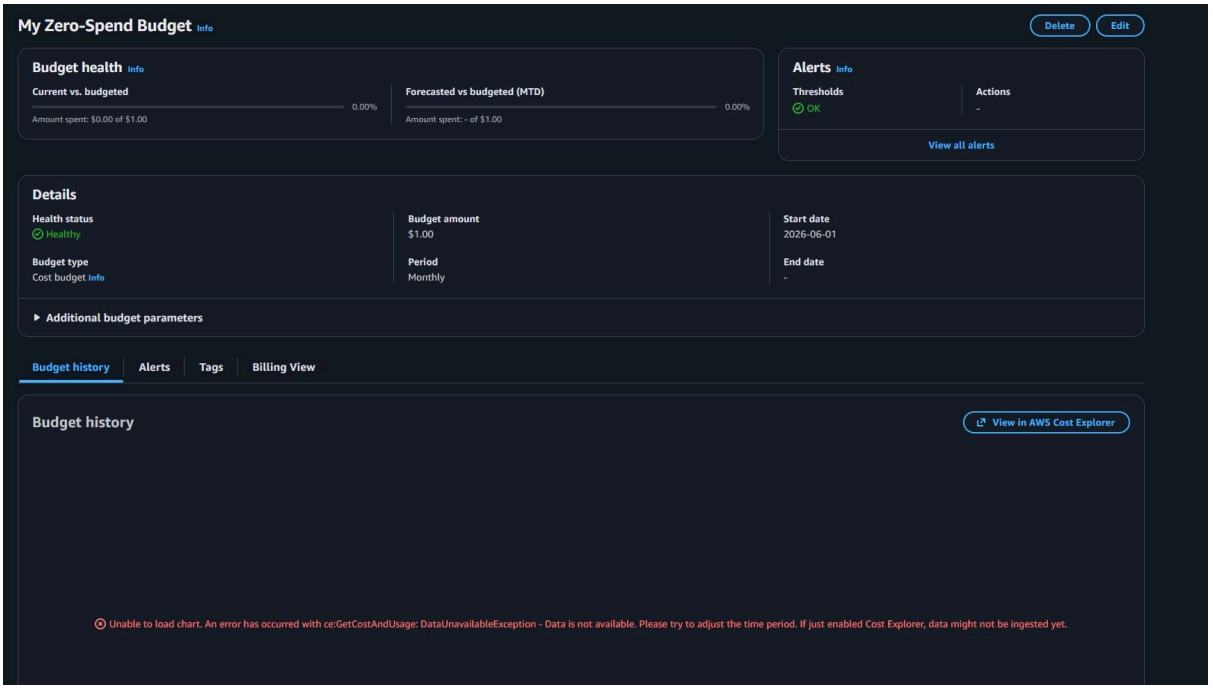


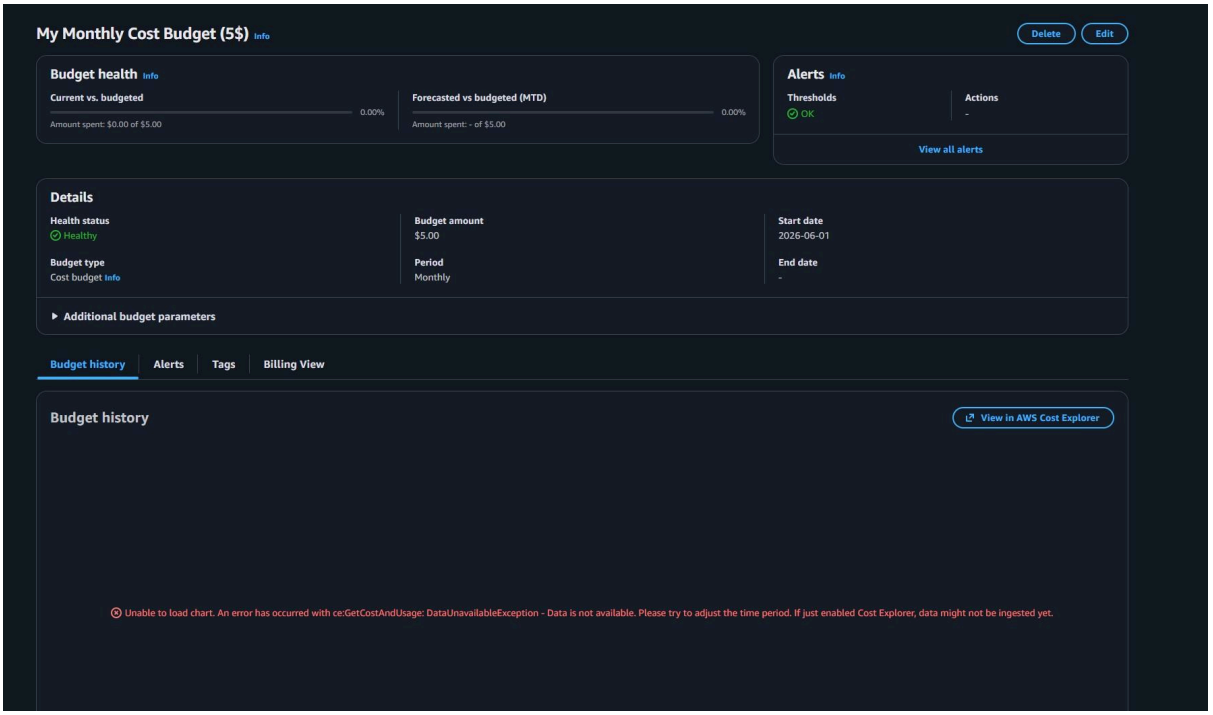
Annexe 7.4 : Erreur conflit de ports



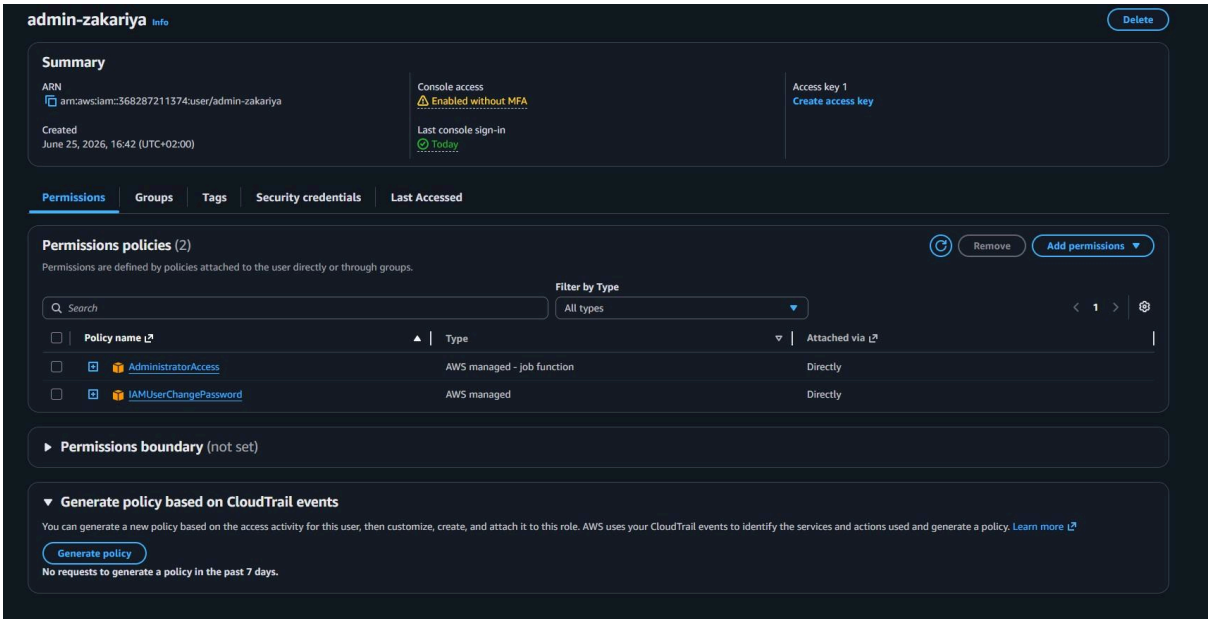
Annexe 7.5 : Application fonctionnelle sur VM



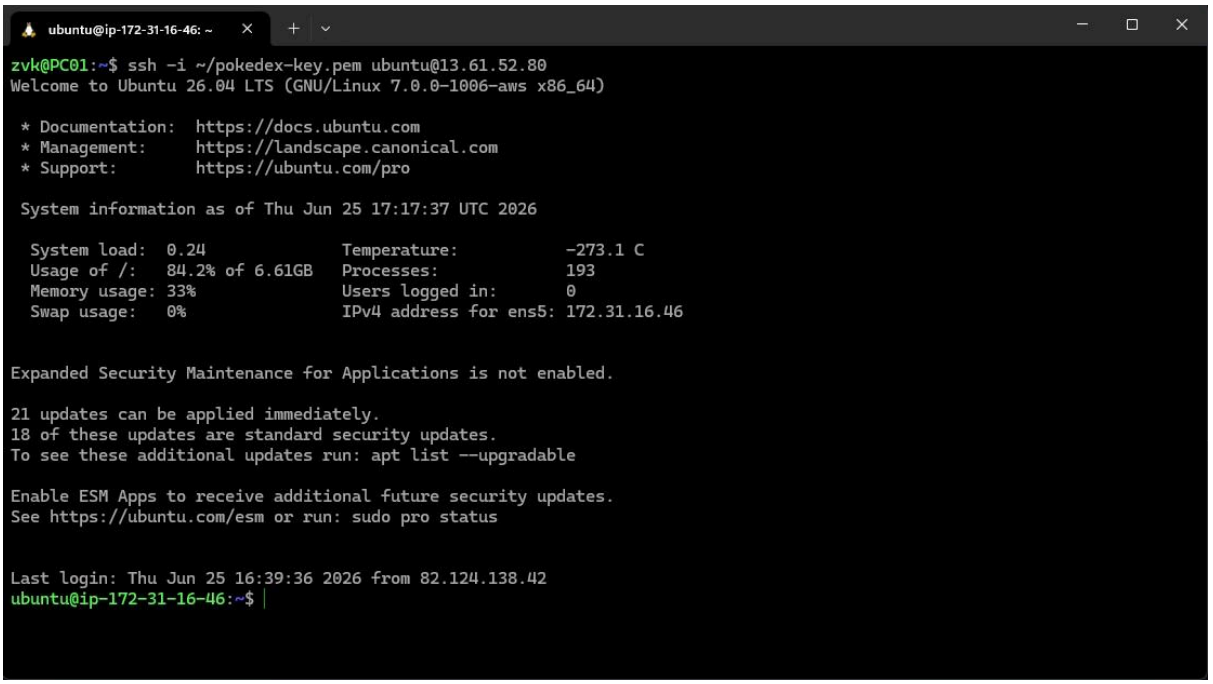

My Zero-Spend Budget info
Delete
Edit
Budget health Info
Current vs. budgeted
0.00%
Amount spent: $0.00 of $1.00
Forecasted vs budgeted (MTD)
0.00%
Amount spent: - of $1.00
Alerts Info
Thresholds
OK
Actions
-
View all alerts
Details
Health status
Healthy
Budget amount
$1.00
Start date
2026-06-01
Budget type
Cost budget Info
Period
Monthly
End date
-
Additional budget parameters
Budget history
Alerts
Tags
Billing View
Budget history
View in AWS Cost Explorer
Unable to load chart. An error has occurred with ce:GetCostAndUsage: DataUnavailableException - Data is not available. Please try to adjust the time period. If just enabled Cost Explorer, data might not be ingested yet.

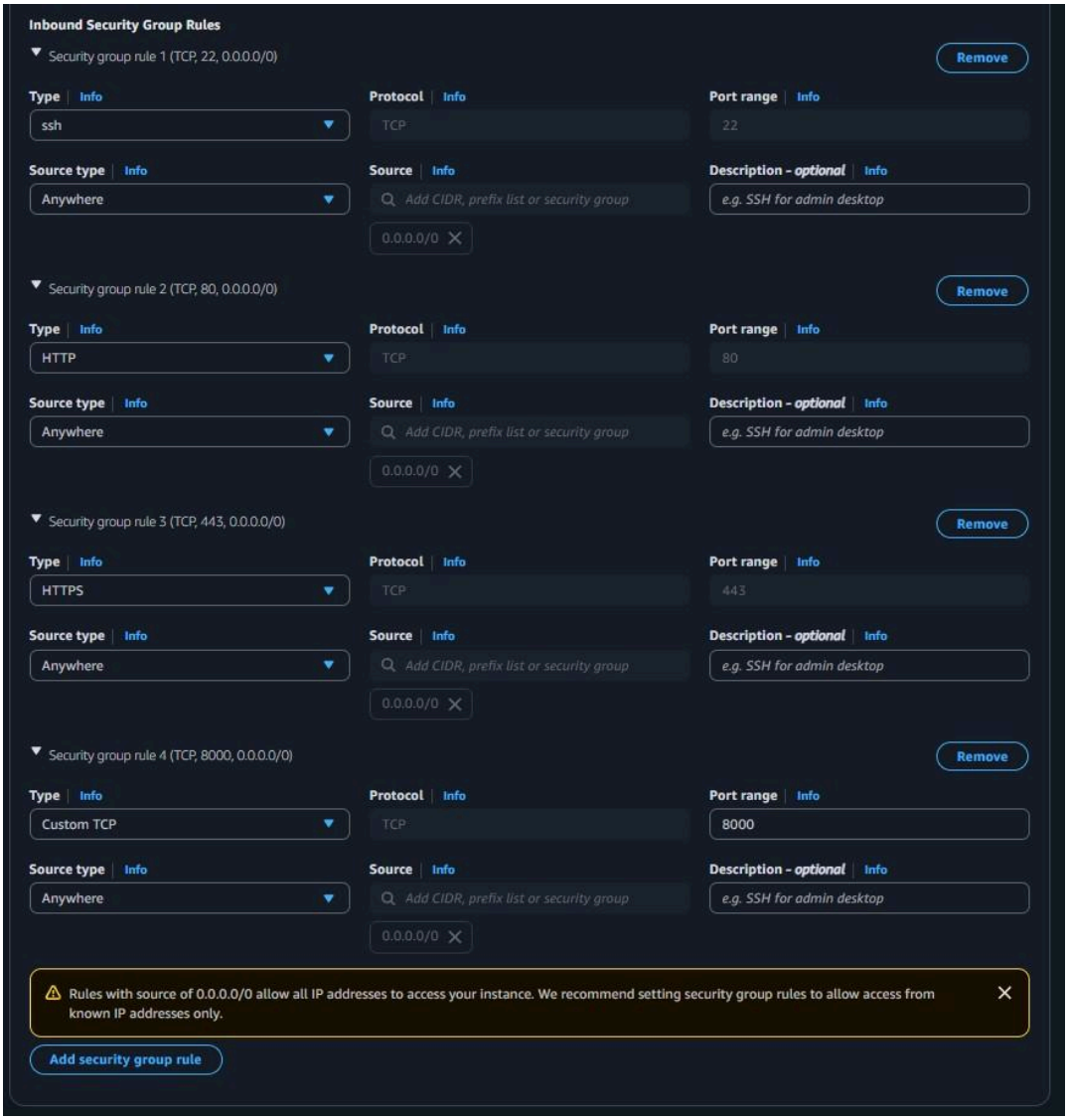

My Monthly Cost Budget (5$) Info
Delete
Edit
Budget health Info
Current vs. budgeted
0.00%
Amount spent: $0.00 of $5.00
Forecasted vs budgeted (MTD)
0.00%
Amount spent: - of $5.00
Alerts Info
Thresholds
OK
Actions
-
View all alerts
Details
Health status
Healthy
Budget amount
$5.00
Start date
2026-06-01
Budget type
Cost budget Info
Period
Monthly
End date
-
Additional budget parameters
Budget history
Alerts
Tags
Billing View
Budget history
View in AWS Cost Explorer
Unable to load chart. An error has occurred with ce:GetCostAndUsage: DataUnavailableException - Data is not available. Please try to adjust the time period. If just enabled Cost Explorer, data might not be ingested yet.

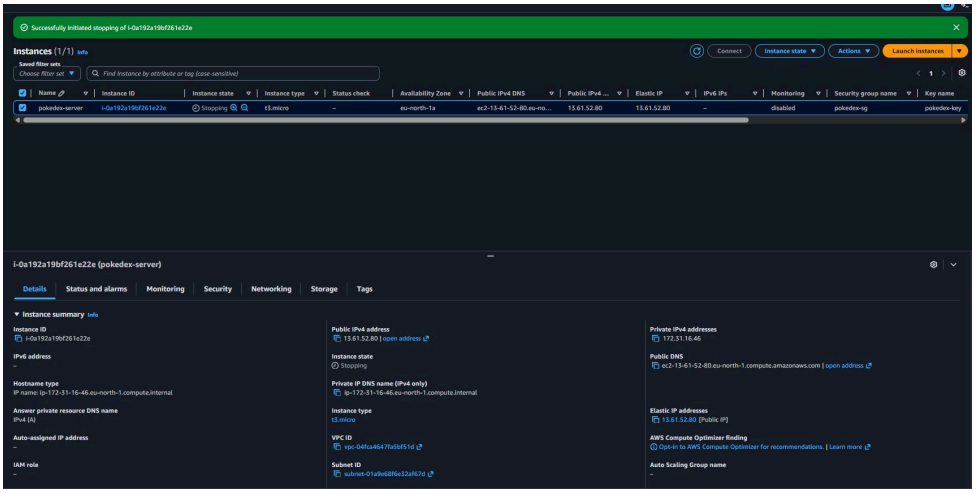

IAM users (1) Info
An IAM user is an identity with long-term credentials that is used to interact with AWS in an account.
Search
Delete
Create user
User name
Path
Groups
Last activity
MFA
Password age
Console last sign-in
Access key ID
Active key age
Access key last use
ARN
Creation time
admin-zakariya
/
0
1 hour ago
-
1 hour
1 hour ago
-
-
-
arn:aws:iam::368287211374:user/adm...
1 hour ago



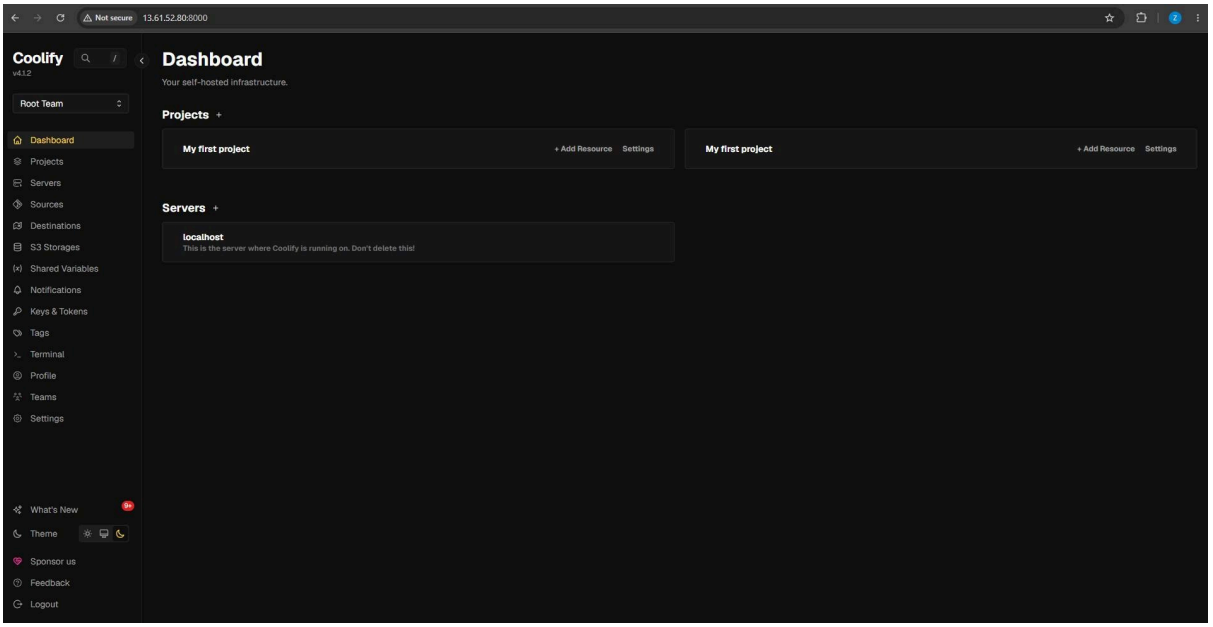
Annexe 8.1 : Alertes de facturation + utilisateur IAM



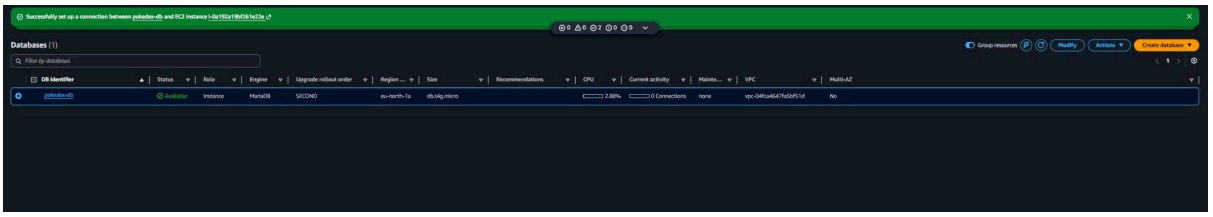


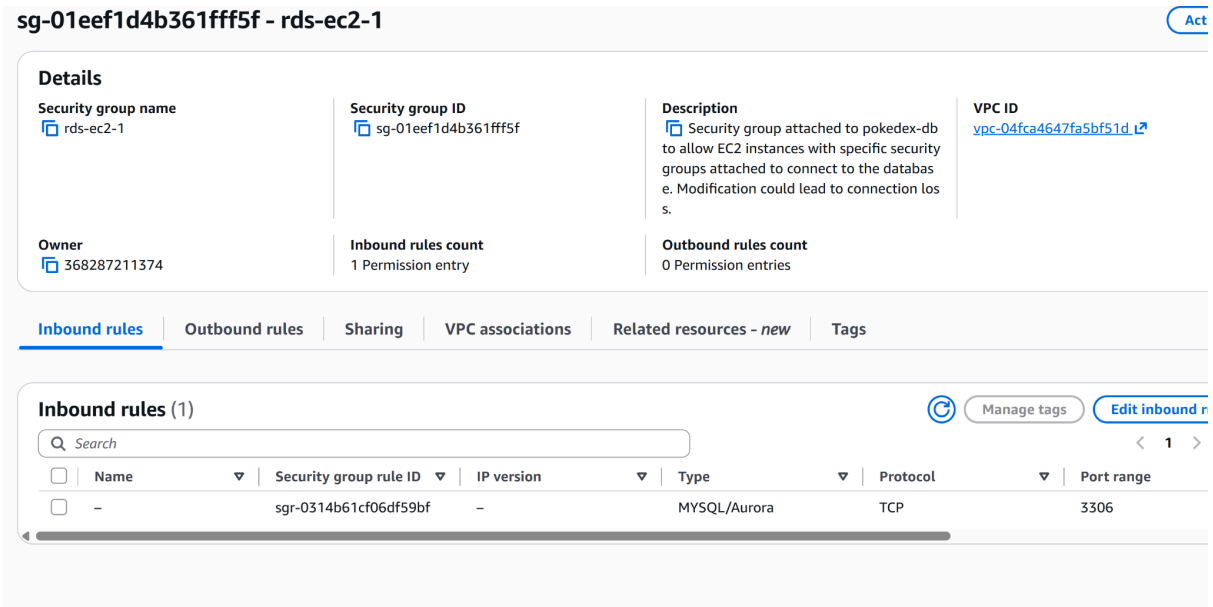


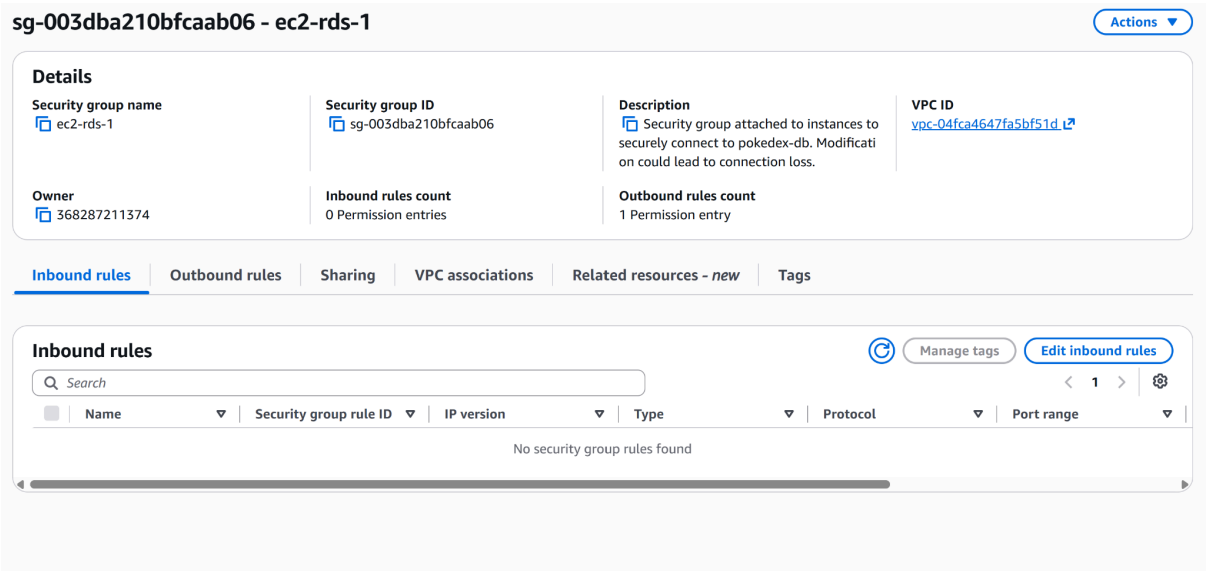
Annexe 8.2 : Instance EC2 Running + Security Groups + Elastic IP



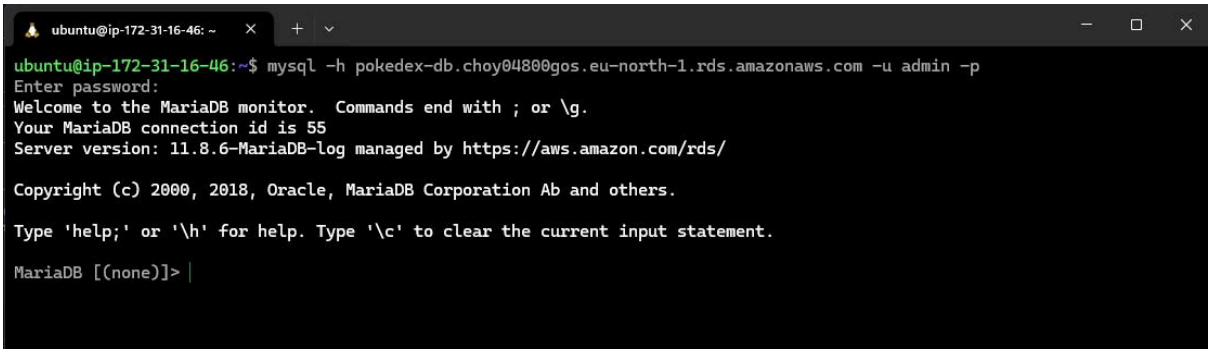
Annexe 8.3 : Dashboard Coolify accessible

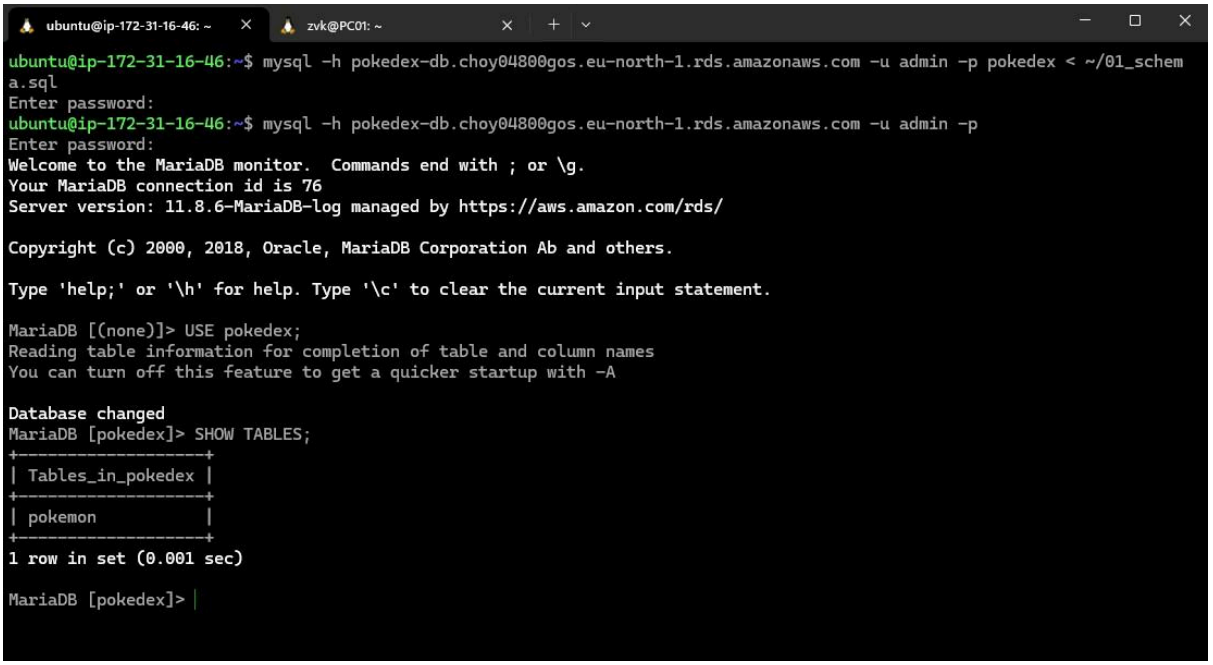






Annexe 8.4 : Instance RDS Available + Security Group





```
ubuntu@ip-172-31-16-46: ~   zvk@PC01: ~

ubuntu@ip-172-31-16-46:~$ mysql -h pokedex-db.choy04800gos.eu-north-1.rds.amazonaws.com -u admin -p pokedex < ~/02_pokem
ons.sql
Enter password:
ubuntu@ip-172-31-16-46:~$
```

```
ubuntu@ip-172-31-16-46: ~   zvk@PC01: ~

ubuntu@ip-172-31-16-46:~$ mysql -h pokedex-db.choy04800gos.eu-north-1.rds.amazonaws.com -u admin -p pokedex
Enter password:
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

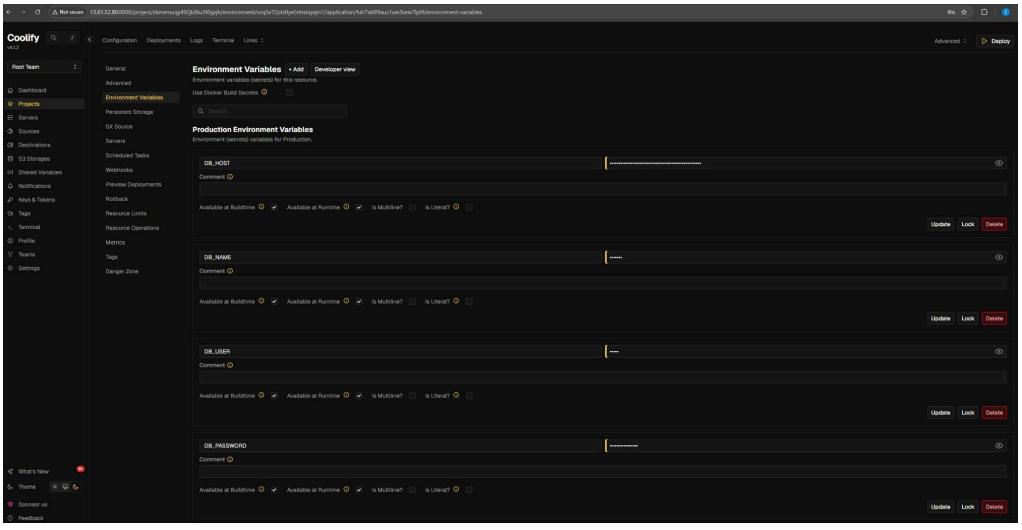
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 228
Server version: 11.8.6-MariaDB-log managed by https://aws.amazon.com/rds/

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

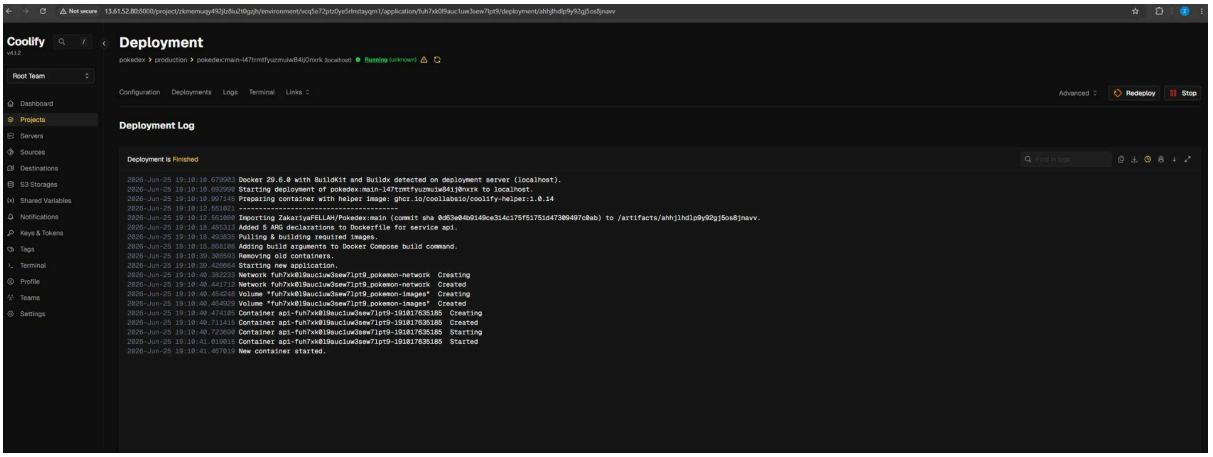
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

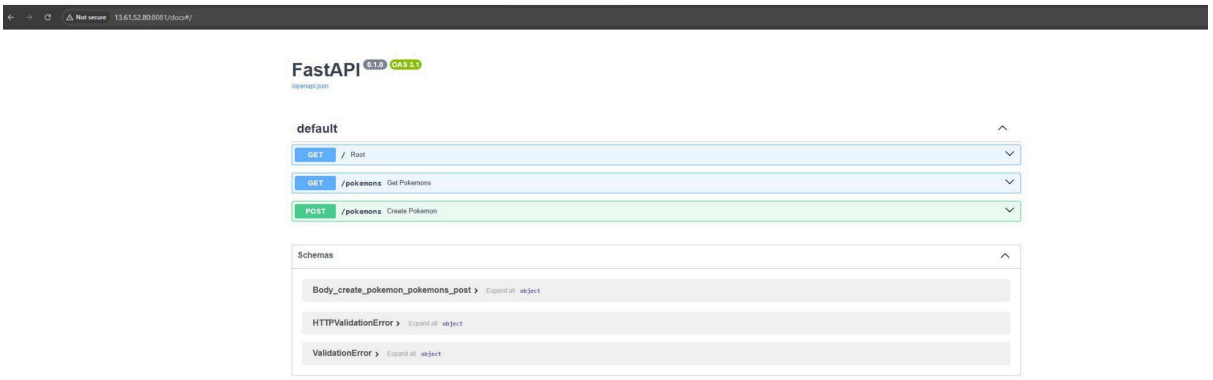
MariaDB [pokedex]> USE pokedex;
Database changed
MariaDB [pokedex]> SELECT * FROM pokemon;
+----+------------+--------------------+-------+------+--------+---------+----------------+-----------------+-------+---
----------+------------------------+
| id | name       | types              | total | hp   | attack | defense | attack_special | defense_special | speed | ev
olution_id | image_url              |
+----+------------+--------------------+-------+------+--------+---------+----------------+-----------------+-------+---
----------+------------------------+
|  1 | Bulbasaur  | ["Grass","Poison"] |   318 |   45 |     49 |      49 |             65 |              65 |    45 |
        2 | /static/images/1.png   |
|  2 | Ivysaur    | ["Grass","Poison"] |   405 |   60 |     62 |      63 |             80 |              80 |    60 |
        3 | /static/images/2.png   |
|  3 | Venusaur   | ["Grass","Poison"] |   525 |   80 |     82 |      83 |            100 |             100 |    80 |
     NULL | /static/images/3.png   |
|  4 | Charmander | ["Fire"]           |   309 |   39 |     52 |      43 |             60 |              50 |    65 |
        5 | /static/images/4.png   |
```

Annexe 8.5 : Connexion SSH → RDS + import schéma/données

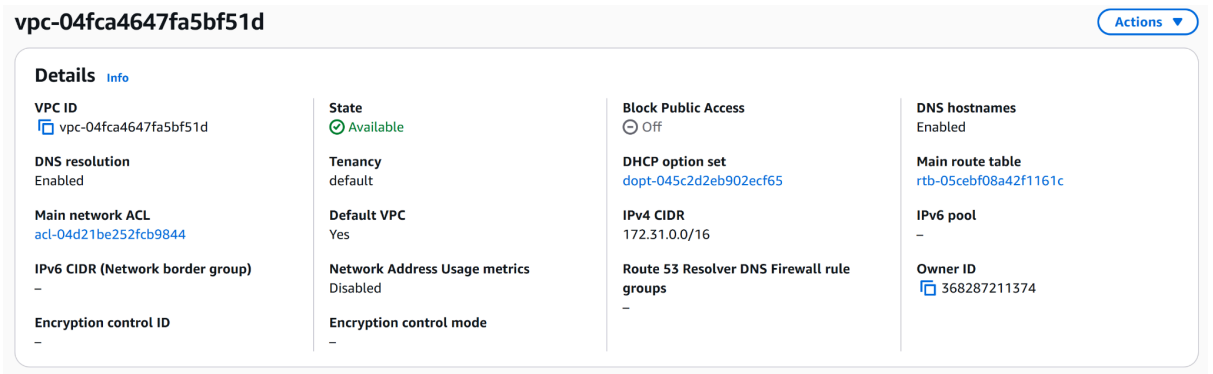


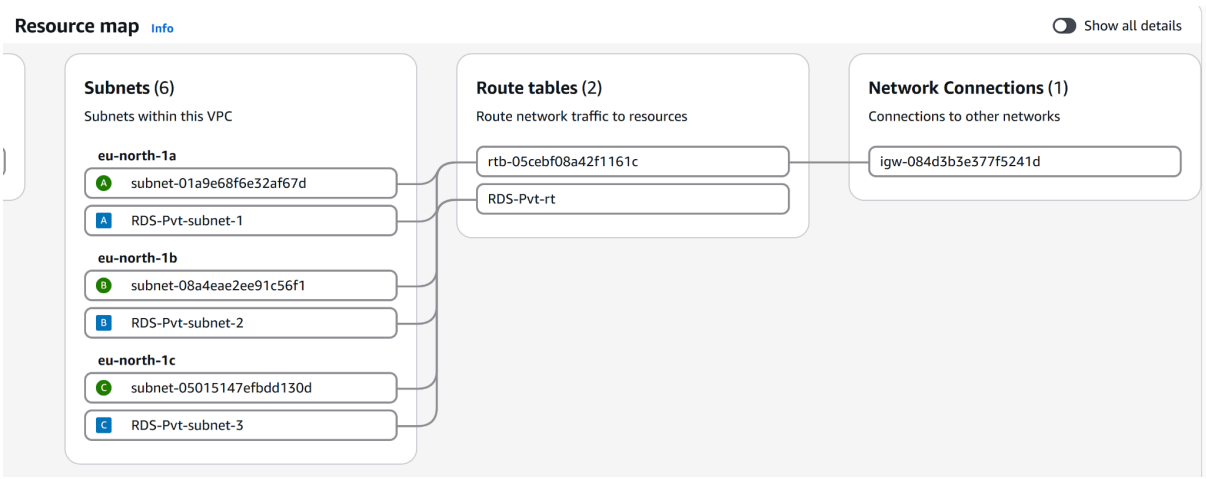
Annexe 8.6 : Variables d'environnement Coolify (RDS)

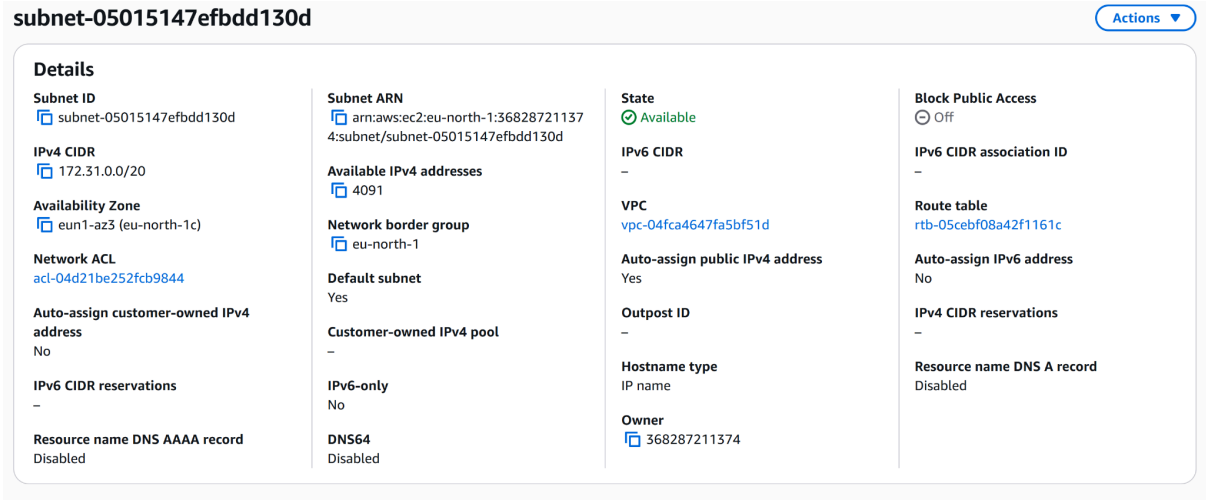


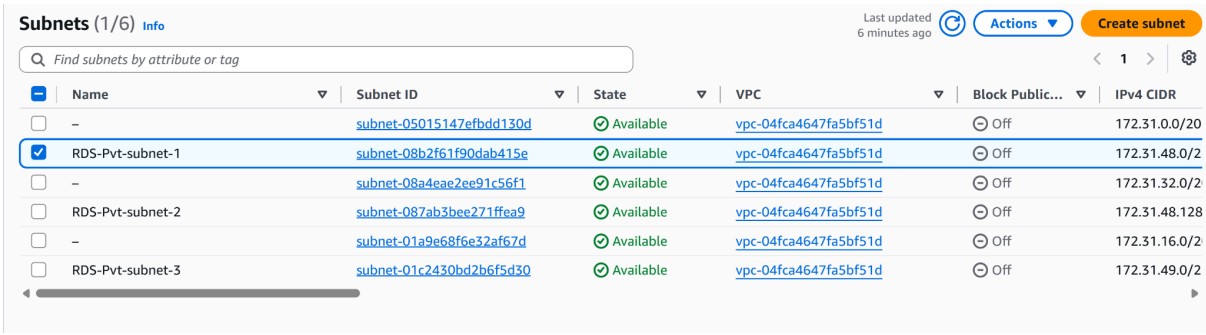


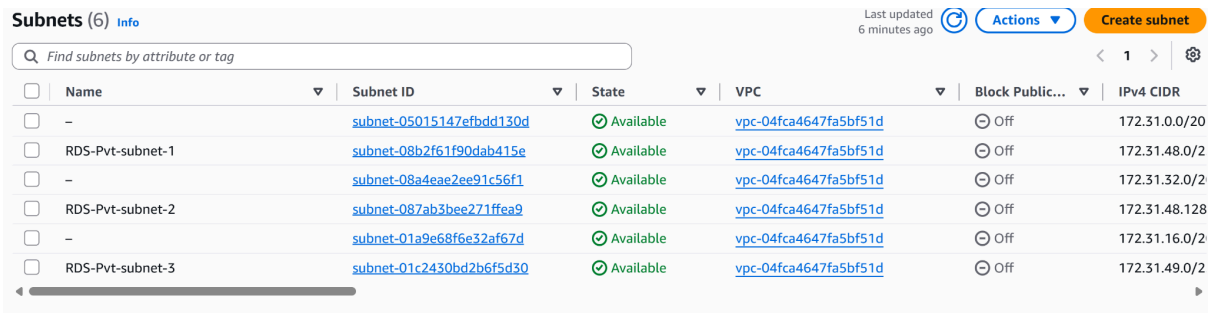
Annexe 8.7 : Déploiement Coolify réussi + API accessible

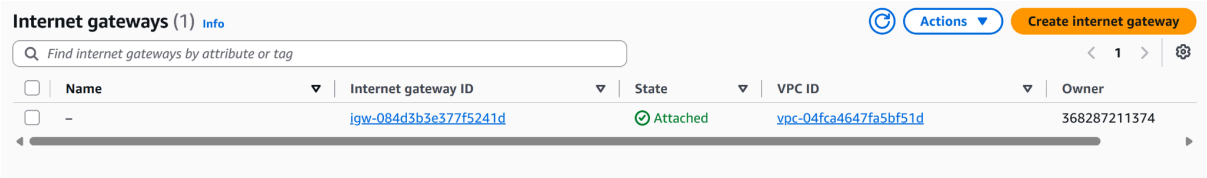



Resource map Info
Show all details
Subnets (6)
Subnets within this VPC
eu-north-1a
subnet-01a9e68f6e32af67d
RDS-Pvt-subnet-1
eu-north-1b
subnet-08a4eae2ee91c56f1
RDS-Pvt-subnet-2
eu-north-1c
subnet-05015147efbdd130d
RDS-Pvt-subnet-3
Route tables (2)
Route network traffic to resources
rtb-05cebf08a42f1161c
RDS-Pvt-rt
Network Connections (1)
Connections to other networks
igw-084d3b3e377f5241d

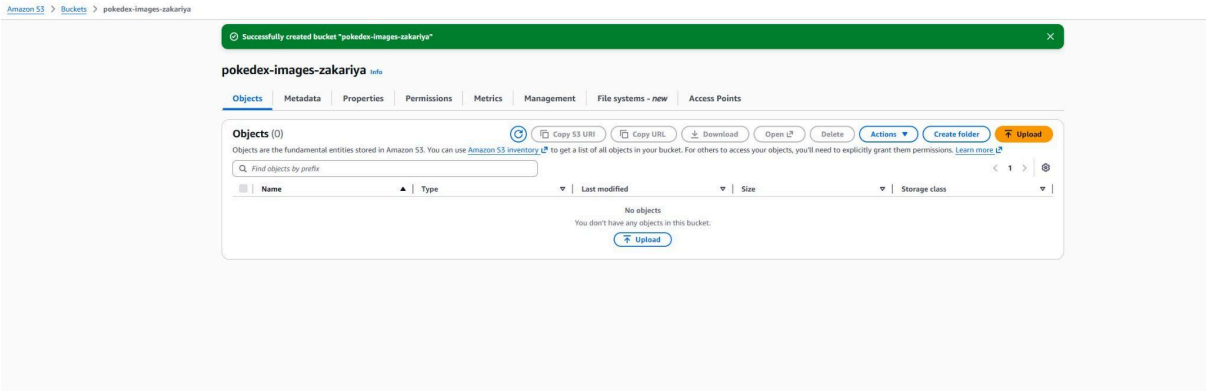

subnet-05015147efbdd130d
Actions
Details
Subnet ID
subnet-05015147efbdd130d
IPv4 CIDR
172.31.0.0/20
Availability Zone
eun1-az3 (eu-north-1c)
Network ACL
acl-04d21be252fcb9844
Auto-assign customer-owned IPv4 address
No
IPv6 CIDR reservations
–
Resource name DNS AAAA record
Disabled
Subnet ARN
arn:aws:ec2:eu-north-1:368287211374:subnet/subnet-05015147efbdd130d
Available IPv4 addresses
4091
Network border group
eu-north-1
Default subnet
Yes
Customer-owned IPv4 pool
–
IPv6-only
No
DNS64
Disabled
State
Available
IPv6 CIDR
–
VPC
vpc-04fca4647fa5bf51d
Auto-assign public IPv4 address
Yes
Outpost ID
–
Hostname type
IP name
Owner
368287211374
Block Public Access
Off
IPv6 CIDR association ID
–
Route table
rtb-05cebf08a42f1161c
Auto-assign IPv6 address
No
IPv4 CIDR reservations
–
Resource name DNS A record
Disabled

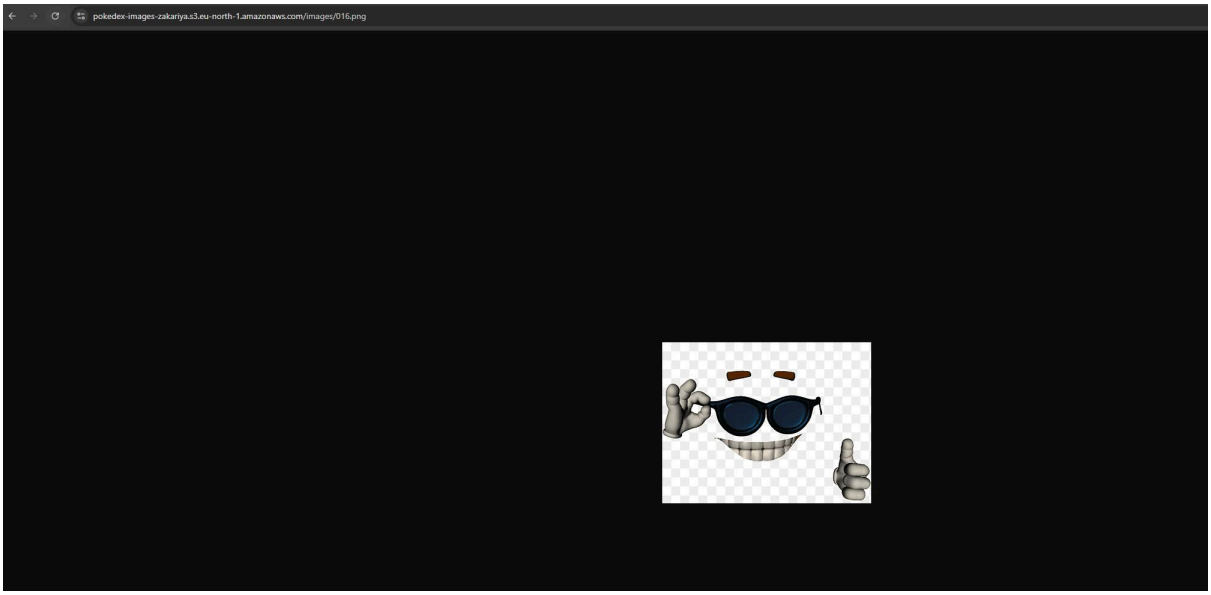

Subnets (1/6) Info
Last updated 6 minutes ago
Actions
Create subnet
Find subnets by attribute or tag
Name
Subnet ID
State
VPC
Block Public...
IPv4 CIDR
– subnet-05015147efbdd130d Available vpc-04fca4647fa5bf51d Off 172.31.0.0/20
RDS-Pvt-subnet-1 subnet-08b2f61f90dab415e Available vpc-04fca4647fa5bf51d Off 172.31.48.0/2
– subnet-08a4eae2ee91c56f1 Available vpc-04fca4647fa5bf51d Off 172.31.32.0/2
RDS-Pvt-subnet-2 subnet-087ab3bee271ffea9 Available vpc-04fca4647fa5bf51d Off 172.31.48.128
– subnet-01a9e68f6e32af67d Available vpc-04fca4647fa5bf51d Off 172.31.16.0/2
RDS-Pvt-subnet-3 subnet-01c2430bd2b6f5d30 Available vpc-04fca4647fa5bf51d Off 172.31.49.0/2


Subnets (6) Info
Last updated 6 minutes ago
Actions
Create subnet
Find subnets by attribute or tag
Name
Subnet ID
State
VPC
Block Public...
IPv4 CIDR
– subnet-05015147efbdd130d Available vpc-04fca4647fa5bf51d Off 172.31.0.0/20
RDS-Pvt-subnet-1 subnet-08b2f61f90dab415e Available vpc-04fca4647fa5bf51d Off 172.31.48.0/2
– subnet-08a4eae2ee91c56f1 Available vpc-04fca4647fa5bf51d Off 172.31.32.0/2
RDS-Pvt-subnet-2 subnet-087ab3bee271ffea9 Available vpc-04fca4647fa5bf51d Off 172.31.48.128
– subnet-01a9e68f6e32af67d Available vpc-04fca4647fa5bf51d Off 172.31.16.0/2
RDS-Pvt-subnet-3 subnet-01c2430bd2b6f5d30 Available vpc-04fca4647fa5bf51d Off 172.31.49.0/2

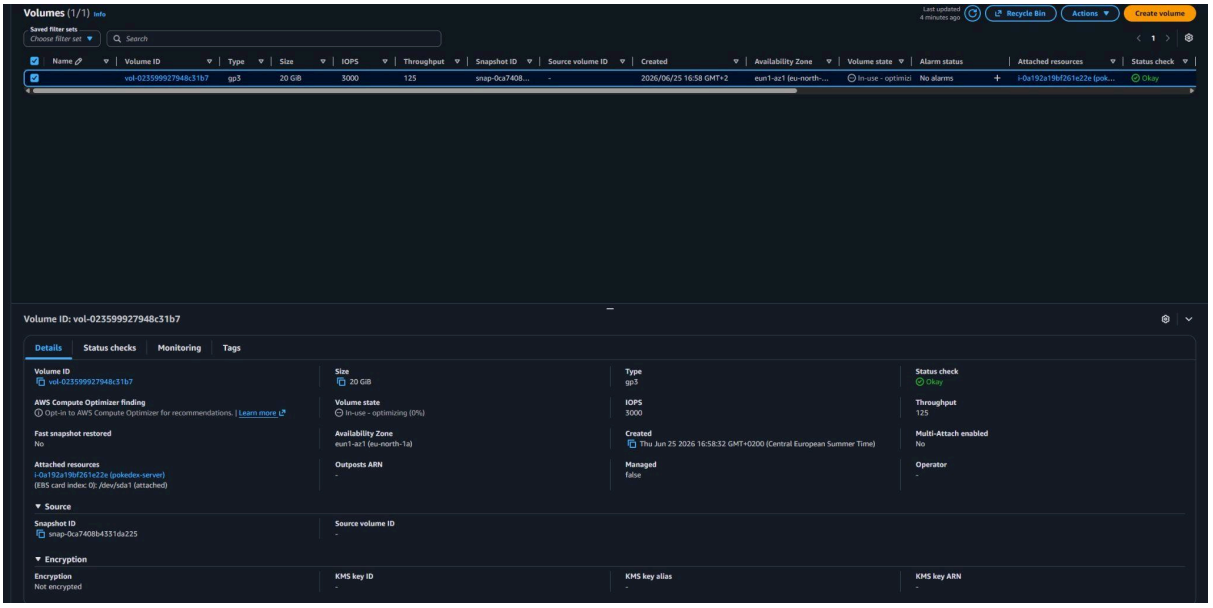


Annexe 8.8 : VPC avec subnets + Internet Gateway

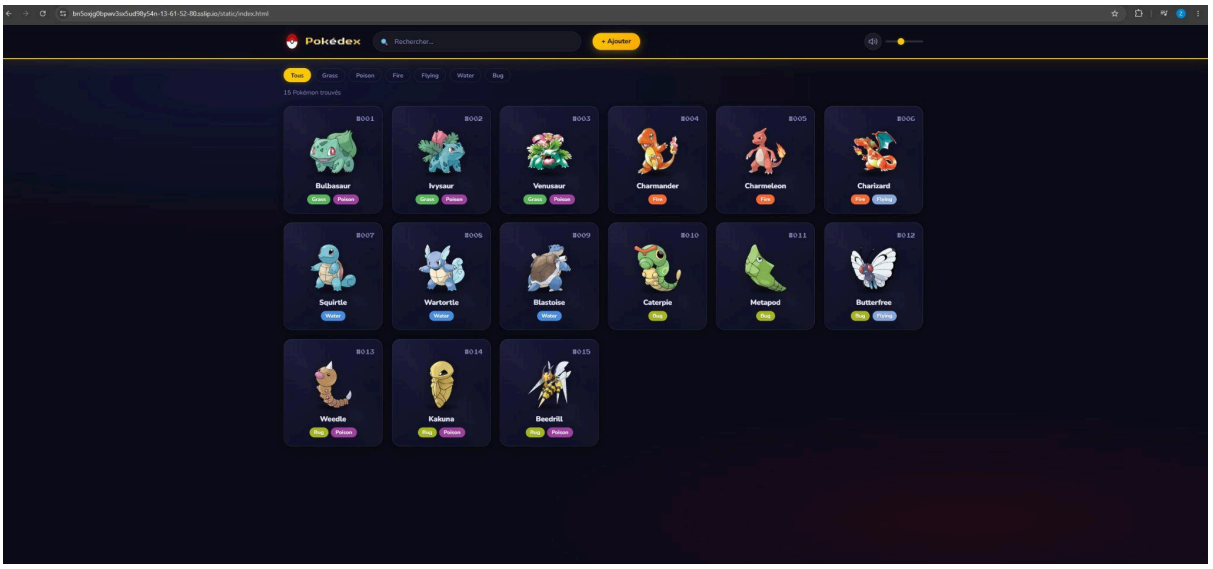




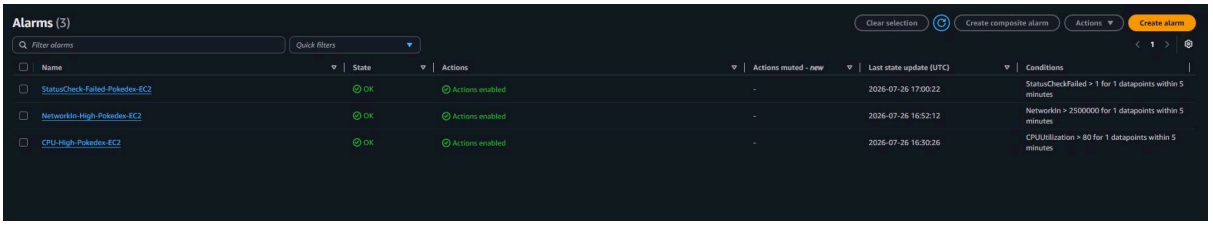
Annexe 8.9 : Bucket S3 + image accessible

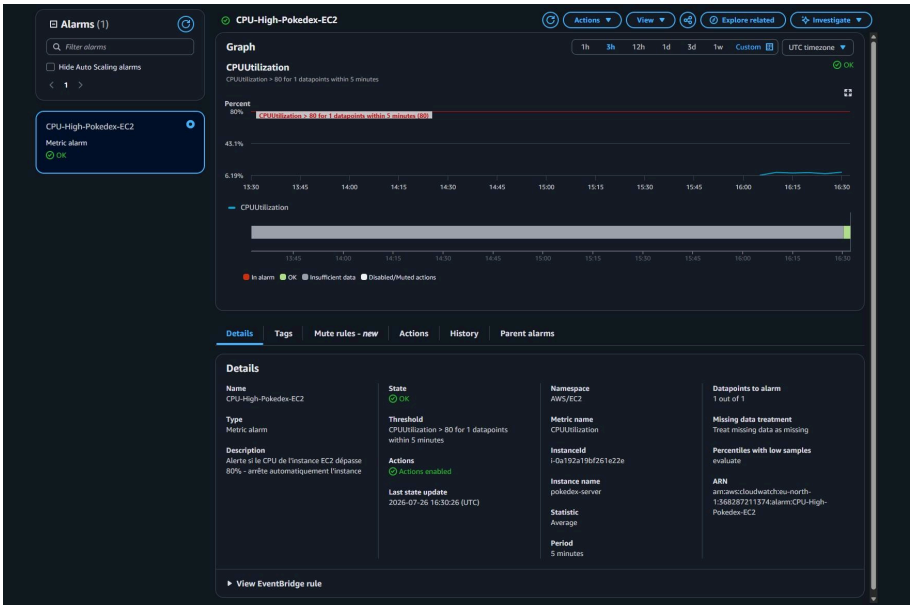


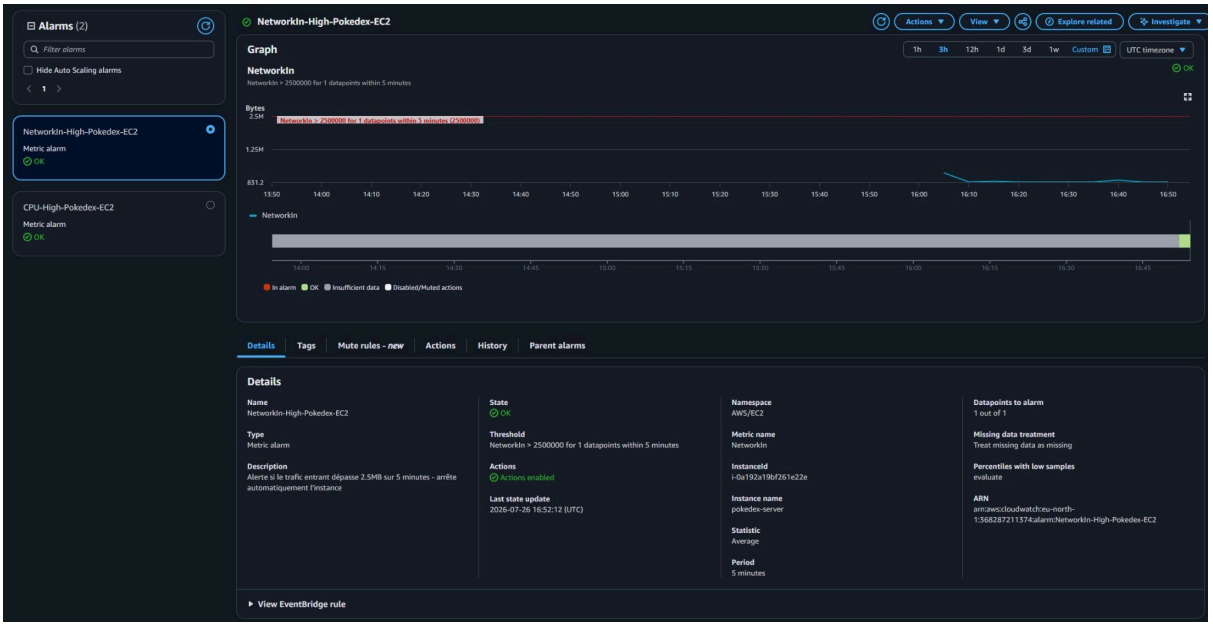
Annexe 9.1 : Extension EBS (8Go → 20Go)

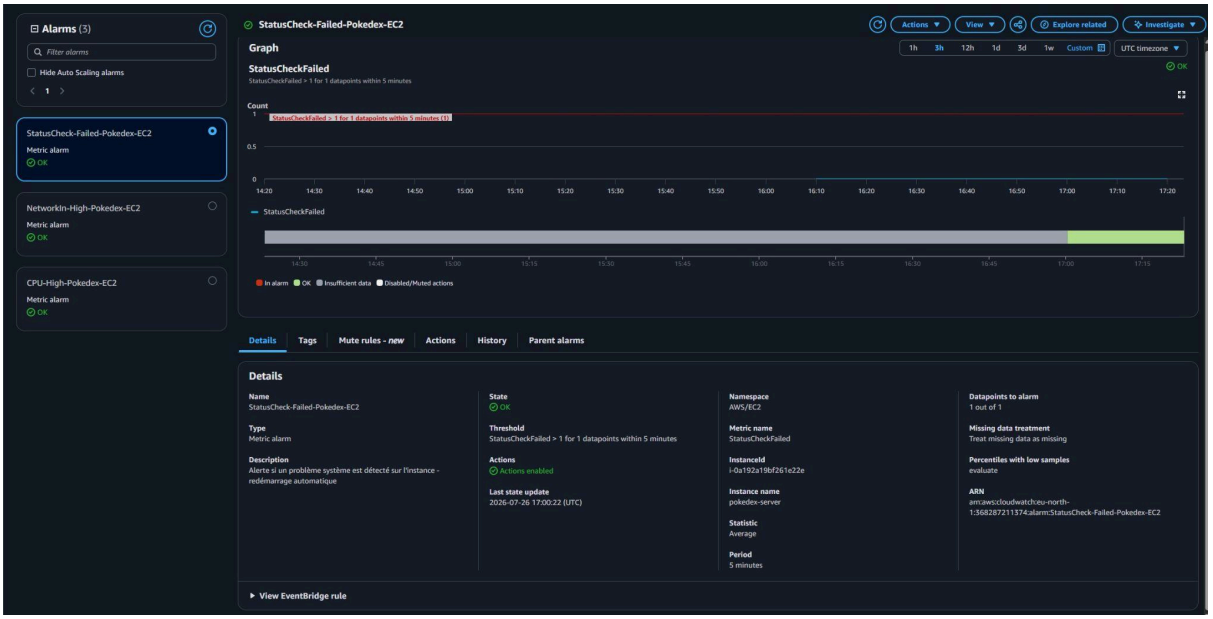


Annexe 9.2 : Application en HTTPS avec cadenas

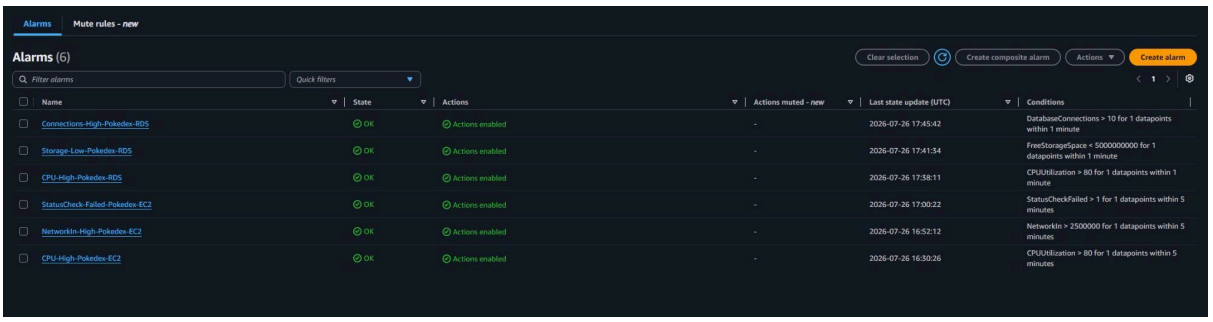


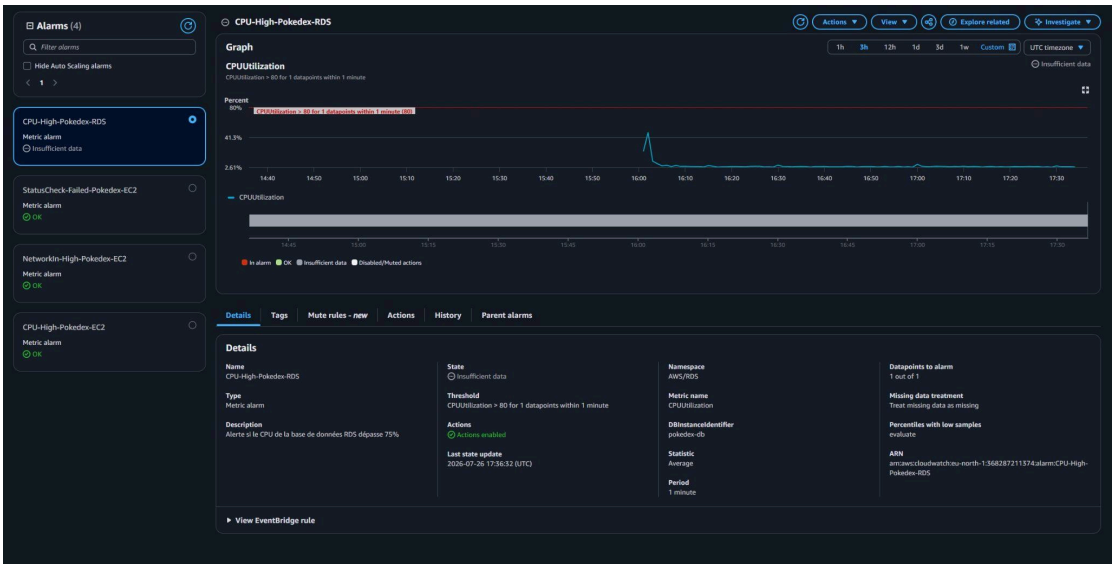

Alarms (1)
Filter alarms
Hide Auto Scaling alarms
CPU-High-Pokedex-EC2
Metric alarm
OK
CPU-High-Pokedex-EC2
Actions
View
Explore related
Investigate
Graph
1h 3h 12h 1d 3d 1w Custom
UTC timezone
CPUUtilization
CPUUtilization > 80 for 1 datapoints within 5 minutes
OK
Percent
80%
CPUUtilization > 80 for 1 datapoints within 5 minutes (80)
43.1%
6.19%
13:30 13:45 14:00 14:15 14:30 14:45 15:00 15:15 15:30 15:45 16:00 16:15 16:30
CPUUtilization
In alarm
OK
Insufficient data
Disabled/Muted actions
Details
Tags
Mute rules - new
Actions
History
Parent alarms
Details
Name
CPU-High-Pokedex-EC2
Type
Metric alarm
Description
Alerte si le CPU de l'instance EC2 dépasse 80% - arrête automatiquement l'instance
State
OK
Threshold
CPUUtilization > 80 for 1 datapoints within 5 minutes
Actions
Actions enabled
Last state update
2026-07-26 16:30:26 (UTC)
Namespace
AWS/EC2
Metric name
CPUUtilization
InstanceId
i-0a192a19bf261e22e
Instance name
pokedex-server
Statistic
Average
Period
5 minutes
Datapoints to alarm
1 out of 1
Missing data treatment
Treat missing data as missing
Percentiles with low samples
evaluate
ARN
arn:aws:cloudwatch:eu-north-1:368287211374:alarm:CPU-High-Pokedex-EC2
View EventBridge rule

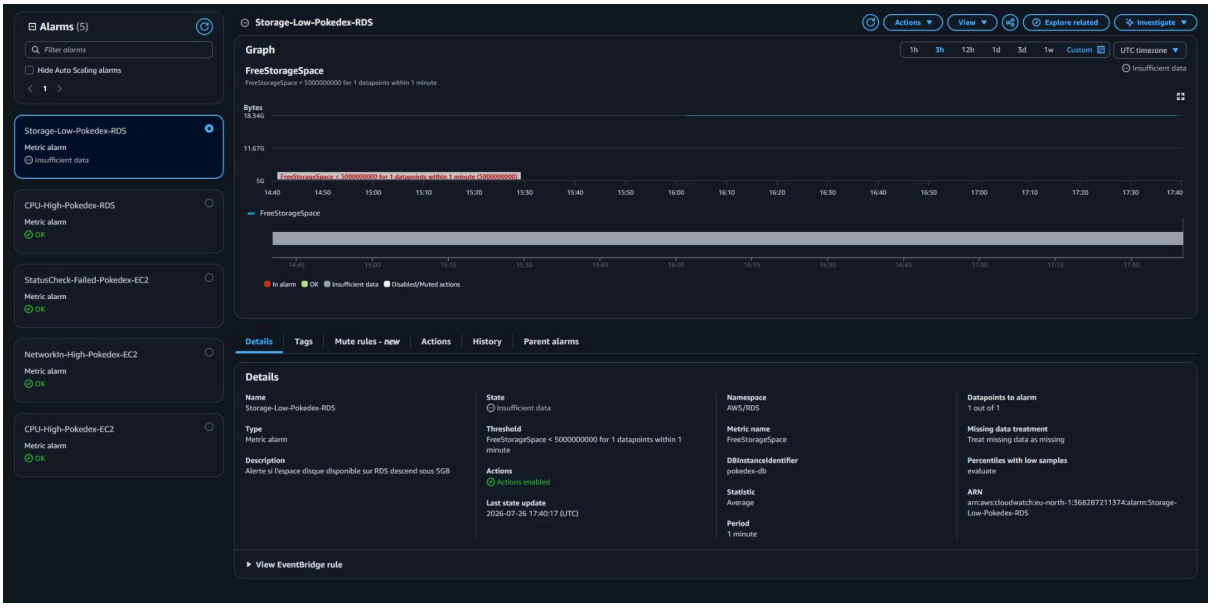

Alarms (2)
Filter alarms
Hide Auto Scaling alarms
NetworkIn-High-Pokedex-EC2
Metric alarm
OK
CPU-High-Pokedex-EC2
Metric alarm
OK
NetworkIn-High-Pokedex-EC2
Actions
View
Explore related
Investigate
Graph
1h 3h 12h 1d 3d 1w Custom
UTC timezone
NetworkIn
NetworkIn > 2500000 for 1 datapoints within 5 minutes
OK
Bytes
2.5M
NetworkIn > 2500000 for 1 datapoints within 5 minutes (2500000)
1.25M
831.2
13:50 14:00 14:10 14:20 14:30 14:40 14:50 15:00 15:10 15:20 15:30 15:40 15:50 16:00 16:10 16:20 16:30 16:40 16:50
NetworkIn
In alarm
OK
Insufficient data
Disabled/Muted actions
Details
Tags
Mute rules - new
Actions
History
Parent alarms
Details
Name
NetworkIn-High-Pokedex-EC2
Type
Metric alarm
Description
Alerte si le trafic entrant dépasse 2.5MB sur 5 minutes - arrête automatiquement l'instance
State
OK
Threshold
NetworkIn > 2500000 for 1 datapoints within 5 minutes
Actions
Actions enabled
Last state update
2026-07-26 16:52:12 (UTC)
Namespace
AWS/EC2
Metric name
NetworkIn
InstanceId
i-0a192a19bf261e22e
Instance name
pokedex-server
Statistic
Average
Period
5 minutes
Datapoints to alarm
1 out of 1
Missing data treatment
Treat missing data as missing
Percentiles with low samples
evaluate
ARN
arn:aws:cloudwatch:eu-north-1:368287211374:alarm:NetworkIn-High-Pokedex-EC2
View EventBridge rule

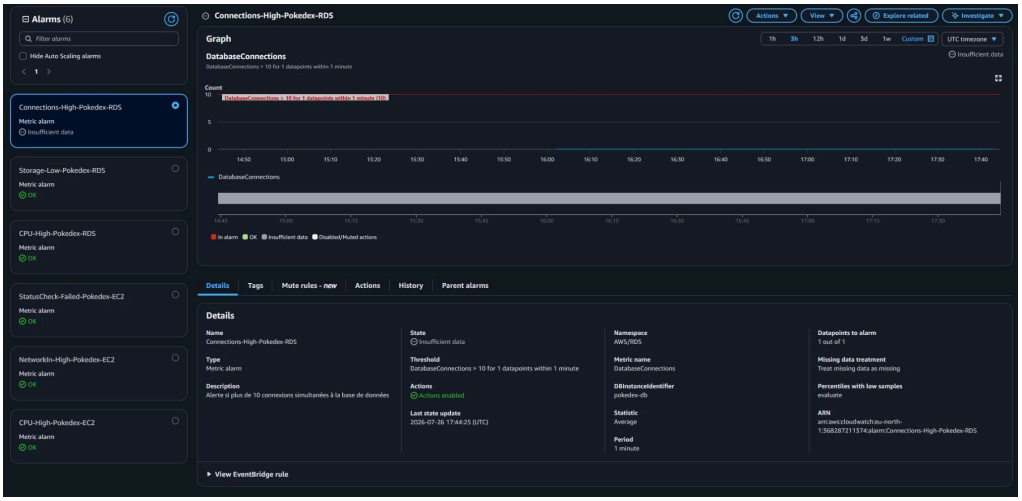


Annexe 9.3 : 3 alarmes CloudWatch EC2





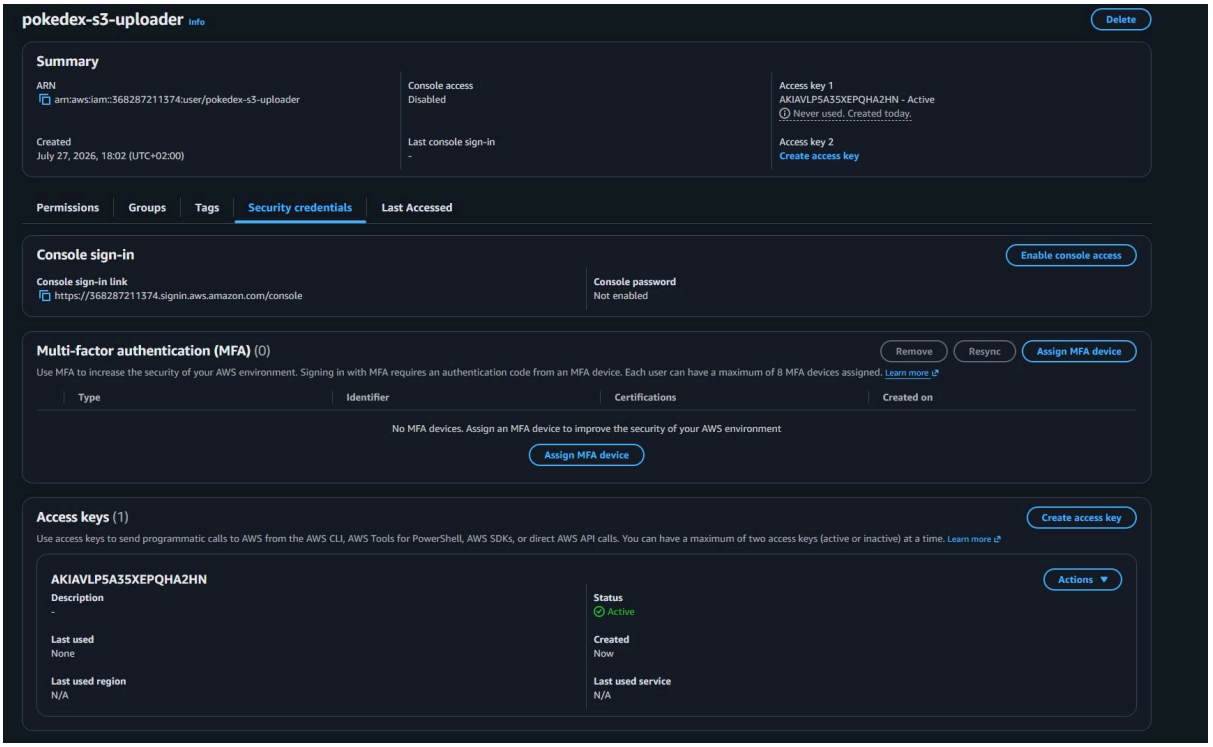


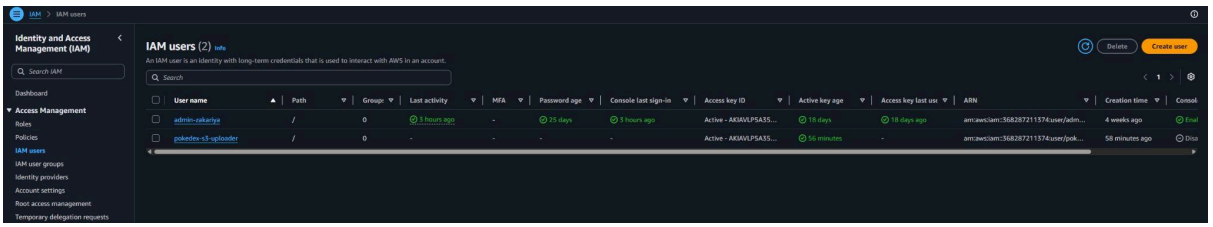


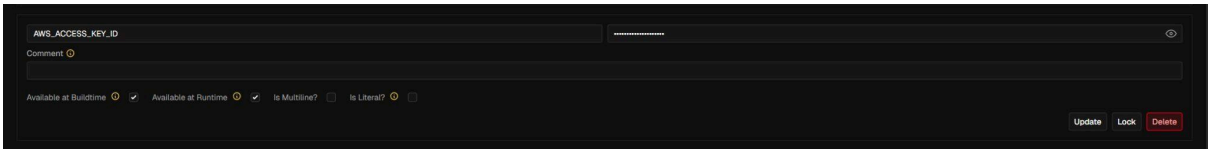
Annexe 9.4 : 3 alarmes CloudWatch RDS

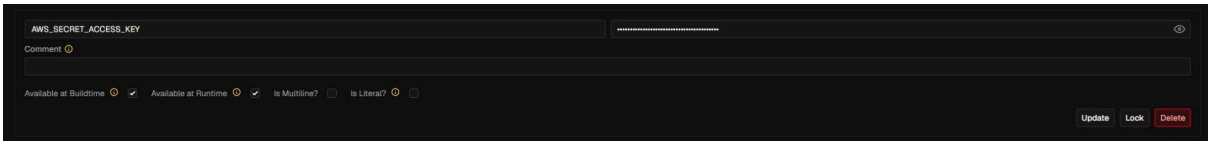


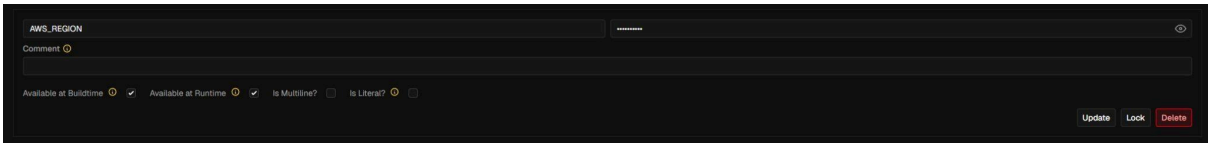
Annexe 9.5 : Dashboard CloudWatch complet

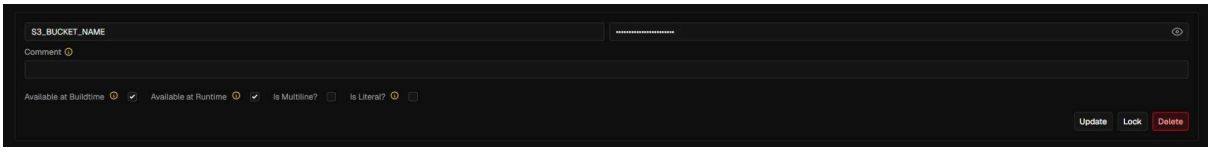

pokedex-s3-uploader Info
Delete
Summary
ARN
arn:aws:iam::368287211374:user/pokedex-s3-uploader
Console access
Disabled
Access key 1
AKIAVLP5A35XEPQHA2HN - Active
Never used. Created today.
Created
July 27, 2026, 18:02 (UTC+02:00)
Last console sign-in
-
Access key 2
Create access key
Permissions
Groups
Tags
Security credentials
Last Accessed
Console sign-in
Enable console access
Console sign-in link
https://368287211374.signin.aws.amazon.com/console
Console password
Not enabled
Multi-factor authentication (MFA) (0)
Remove
Resync
Assign MFA device
Use MFA to increase the security of your AWS environment. Signing in with MFA requires an authentication code from an MFA device. Each user can have a maximum of 8 MFA devices assigned. Learn more
Type
Identifier
Certifications
Created on
No MFA devices. Assign an MFA device to improve the security of your AWS environment
Assign MFA device
Access keys (1)
Create access key
Use access keys to send programmatic calls to AWS from the AWS CLI, AWS Tools for PowerShell, AWS SDKs, or direct AWS API calls. You can have a maximum of two access keys (active or inactive) at a time. Learn more
AKIAVLP5A35XEPQHA2HN
Actions
Description
-
Status
Active
Last used
None
Created
Now
Last used region
N/A
Last used service
N/A


IAM
IAM users
Identity and Access Management (IAM)
Search IAM
Dashboard
Access Management
Roles
Policies
IAM users
IAM user groups
Identity providers
Account settings
Root access management
Temporary delegation requests
IAM users (2) Info
An IAM user is an identity with long-term credentials that is used to interact with AWS in an account.
Search
Delete
Create user
User name
Path
Groups
Last activity
MFA
Password age
Console last sign-in
Access key ID
Active key age
Access key last use
ARN
Creation time
admin-zakariya
pokedex-s3-uploader

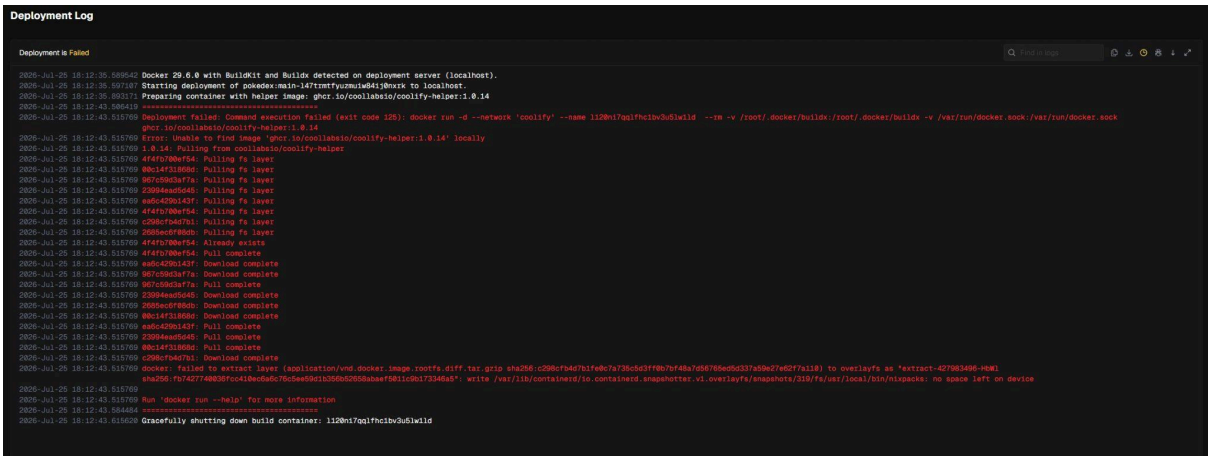

AWS_ACCESS_KEY_ID
Comment
Available at Buildtime
Available at Runtime
Is Multiline?
Is Literal?
Update
Lock
Delete


AWS_SECRET_ACCESS_KEY
Comment
Available at Buildtime
Available at Runtime
Is Multiline?
Is Literal?
Update
Lock
Delete

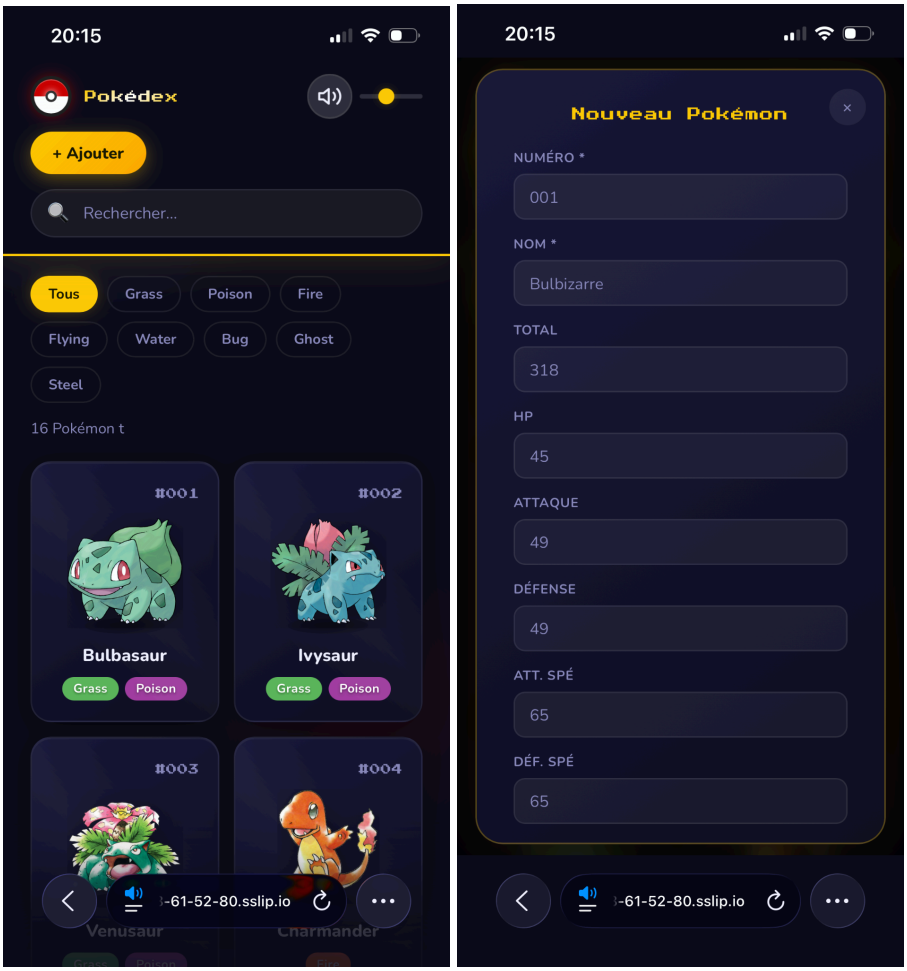

AWS_REGION
Comment
Available at Buildtime
Available at Runtime
Is Multiline?
Is Literal?
Update
Lock
Delete

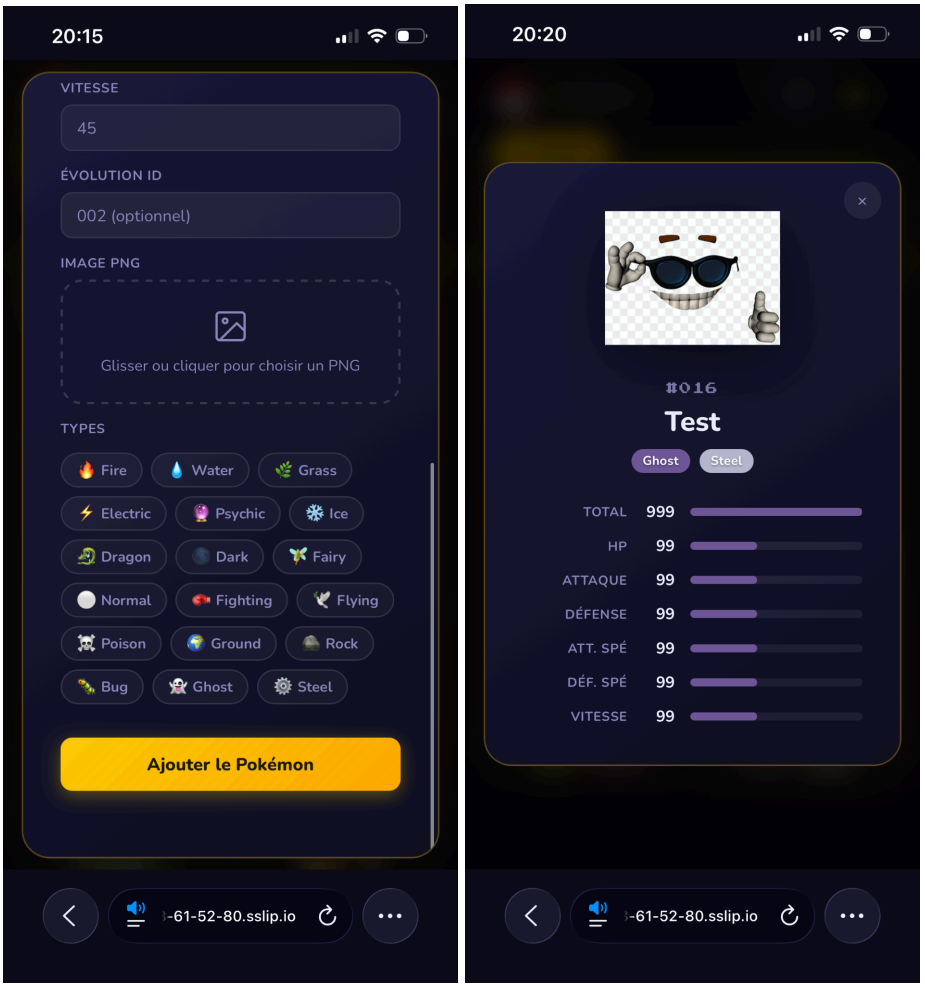


Annexe 9.6 : Utilisateur IAM S3 dédié + variables Coolify



Annexe 9.7 : Logs d'erreur (ModuleNotFoundError, NameError)





Annexe 9.8 : Application fonctionnelle sur mobile

MCD/MLD - Base de données Pokédex

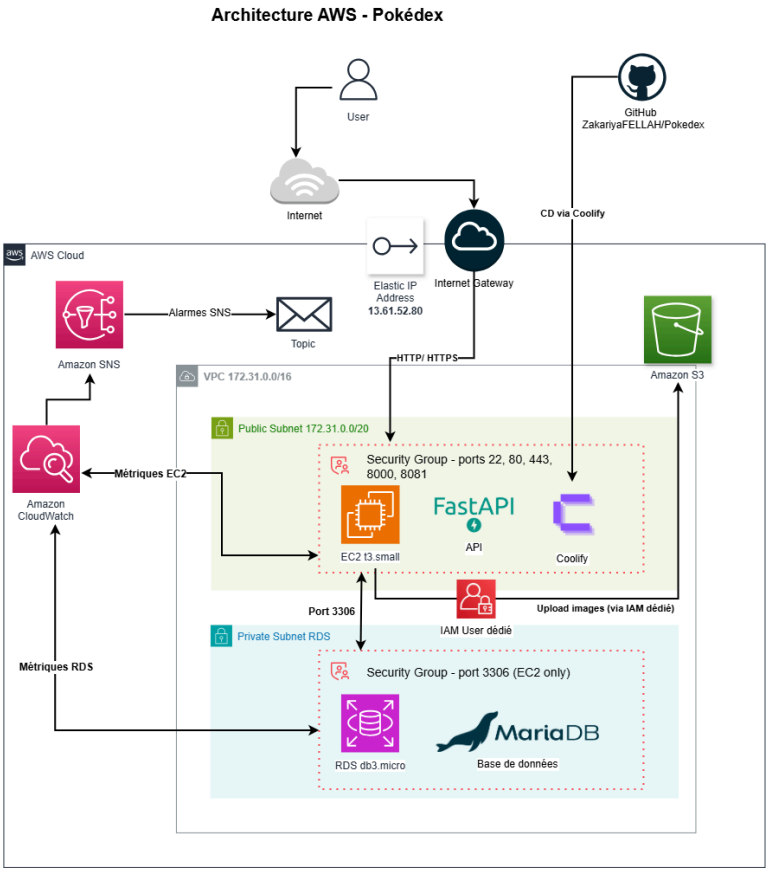
MCD

POKEMON
Identifiant
Nom
Types
Total
PV
Attaque
Défense
Attaque Spéciale
Défense Spéciale
Vitesse
ID Évolution
Image

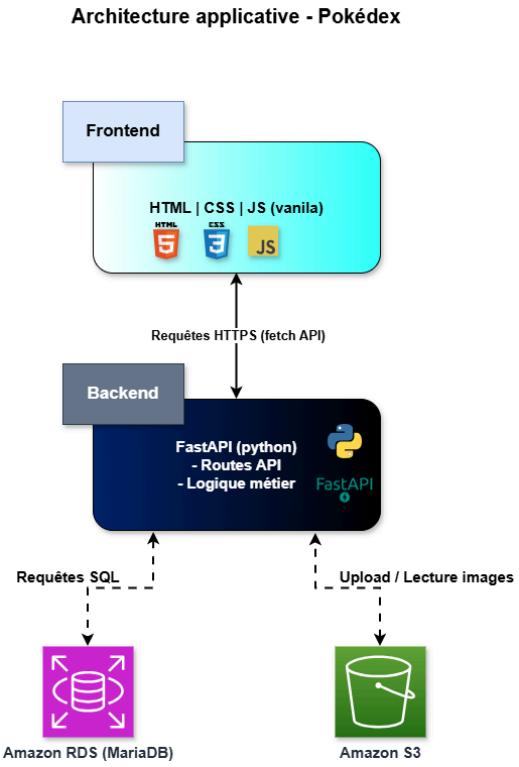
MLD

POKEMON
id INT PRIMARY KEY
name VARCHAR(100)
types JSON
total INT
hp INT
attack INT
defense INT
attack_special INT
defense_special INT
speed INT
evolution_id INT
image_url VARCHAR(255)

Annexe 9.9 : Schéma MCD/MLD



Annexe 9.10 : Schéma d'architecture AWS mis à jour



Annexe 9.11 : Schéma applicatif (Frontend/Backend/BDD/S3)